ACCIDENT DETECTION AND ALERT SYSTEM USING SMARTPHONE SENSORS

A MINI PROJECT REPORT

Submitted by

ABHISHIKTH S SIBY

MLM25MCA-2004

To

the APJ Abdul Kalam Technological University in
Partial fulfillment of the requirements for the
award of the degree

of

MASTER OF COMPUTER APPLICATIONS

Under the Guidance of

Ms. ANJU JOHN

(Assistant Professor, Department of Computer Applications)

DEPARTMENT OF COMPUTER APPLICATIONS

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR

(Affiliated to APJ Abdul Kalam Technological University)

OCTOBER 2026

VISION

To become a centre of excellence in computer applications,competent in the global ecosystem with technical knowledge,innovation with a sense of social commitment.

MISSION

- To serve with state of the art education,foster advanced research and cultivate innovation in the field of computer applications.
- To prepare learners with knowledge skills and critical thinking to excel in the technological landscape and contribute positively to society.

Program Educational Objectives

- PEO I :Graduates will possess a solid foundation and in-depth understanding of computer applications and will be equipped to analyze real-world problems, design and create innovative solutions, and effectively manage and maintain these solutions in their professional careers.
- PEO II: Graduates will acquire technological advancements through continued education, lifelong learning and research, thereby making meaningful contributions to the field of computing.
- PEO III: Graduates will cultivate team spirit, leadership, communication skills, ethics, and social values, enabling them to apply their understanding of the societal impacts of computer applications effectively.

Program Specific Outcomes

- **PSO I**: Apply advanced technologies through innovations to enhance the efficiency of design and development.
- **PSO II**: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

MAPPING OF PO-PSO-SDG

1. MAPPING WITH PROGRAM OUTCOMES (POs):

SL.NO	POs ADDRESSED	RELEVANCE TO PROJECT
1	PO1 - Foundation Knowledge	Uses programming logic, mathematical magnitude calculation and mobile development fundamentals for sensor-based accident detection.
2	PO2 - Problem Analysis	Analyses delayed emergency response, false alerts, device permissions and sensor-threshold problems.
3	PO3 - Development of Solutions	Develops a practical smartphone safety application combining sensors, GPS, alarm and SMS.
4	PO4 - Modern Tool Usage	Applies Flutter, Dart, sensors_plus, geolocator, SharedPreferences and Android SDK tools.

LIST OF PROGRAM OUTCOMES (POs):

PO1 (Foundation Knowledge): Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.

PO2 (Problem Analysis): Identify, review, formulate and analyze problems for primarily focusing on customer requirements using critical thinking frameworks.

PO3 (Development of Solutions): Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.

PO4 (Modern Tool Usage): Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases.

PO5 (Individual and Teamwork): Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.

PO6 (Project Management and Finance): Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of Finance for profitable project management.

PO7 (Ethics): Commit to professional ethics in managing software projects with financial aspects. Learn to use new technologies for cyber security and insulate customers from malware.

PO8 (Life-long learning): Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

2. MAPPING WITH PROGRAM SPECIFIC OUTCOMES (PSOs):

SL.NO	PSOs ADDRESSED	RELEVANCE TO PROJECT
1	PSO 1	Applies smartphone sensing and modern mobile frameworks to build an innovative safety application.
2	PSO 2	Designs and implements a sustainable, low-cost solution to a real-world emergency-response problem.

LIST OF PROGRAM SPECIFIC OUTCOMES (PSOs):

PSO 1: Apply advanced technologies through innovations to enhance the efficiency of design development.

PSO 2: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

3. MAPPING WITH SUSTAINABLE DEVELOPMENT GOALS (SDGs):

SDG NO	SDGs ADDRESSED	RELEVANCE TO PROJECT
3	Good Health and Well-Being	Faster detection and emergency notification can support timely medical assistance after accidents.
9	Industry, Innovation and Infrastructure	Demonstrates innovative use of smartphone infrastructure and embedded sensors for safety.
11	Sustainable Cities and Communities	Supports safer mobility and more responsive urban and rural transport environments.

SUSTAINABLE DEVELOPMENT GOALS (SDGs):

SDG 1 – No Poverty: End poverty in all its forms everywhere.

SDG 2 – Zero Hunger: End hunger, achieve food security and improved nutrition, and promote sustainable agriculture.

SDG 3 – Good Health and Well-Being: Ensure healthy lives and promote well-being for all at all ages.

SDG 4 – Quality Education: Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.

SDG 5 – Gender Equality: Achieve gender equality and empower all women and girls.

SDG 6 – Clean Water and Sanitation: Ensure availability and sustainable management of water and sanitation for all.

SDG 7 – Affordable and Clean Energy: Ensure access to affordable, reliable, sustainable, and modern energy for all.

SDG 8 – Decent Work and Economic Growth: Promote sustained, inclusive, and sustainable economic growth, full and productive employment, and decent work for all.

SDG 9 – Industry, Innovation, and Infrastructure: Build resilient infrastructure, promote inclusive and sustainable industrialization, and foster innovation.

SDG 10 – Reduced Inequality: Reduce inequality within and among countries.

SDG 11 – Sustainable Cities and Communities: Make cities and human settlements inclusive, safe, resilient, and sustainable.

SDG 12 – Responsible Consumption and Production: Ensure sustainable consumption and production patterns.

SDG 13 – Climate Action: Take urgent action to combat climate change and its impacts.

SDG 14 – Life Below Water: Conserve and sustainably use the oceans, seas, and marine resources.

SDG 15 – Life on Land: Protect, restore, and promote sustainable use of terrestrial ecosystems, manage forests sustainably, combat desertification, halt and reverse land degradation, and halt biodiversity loss.

SDG 16 – Peace, Justice, and Strong Institutions: Promote peaceful and inclusive societies, provide access to justice for all, and build effective, accountable, and inclusive institutions.

SDG 17 – Partnerships for the Goals: Strengthen the means of implementation and revitalize the global partnership for sustainable development.

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR
DEPARTMENT OF COMPUTER APPLICATIONS
OCTOBER 2026



DECLARATION

I hereby certify that the work presented in the project entitled "ACCIDENT DETECTION AND ALERT SYSTEM USING SMARTPHONE SENSORS" submitted to the DEPARTMENT OF COMPUTER APPLICATIONS is an authentic record of my own work carried out under the supervision of Ms. Anju John, Assistant Professor. This work has not been submitted to any other institution or university for the award of any other degree. The report has been checked for plagiarism by the college and the similarity index is within the permissible limit set by the college.

Name & Signature of Student

ABHISHIKTH S SIBY

Date:

Place:

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR
DEPARTMENT OF COMPUTER APPLICATIONS
OCTOBER 2026



CERTIFICATE

*Certified that the report entitled "**ACCIDENT DETECTION AND ALERT SYSTEM USING SMARTPHONE SENSORS**" submitted by **ABHISHIKTH S SIBY (MLM25MCA-2004)** to the **APJ Abdul Kalam Technological University** in partial fulfillment of the requirements for the award of **the Degree of Master of Computer Applications** is a bonafide record of the MINI PROJECT work carried out by the student under our guidance and supervision during the academic year 2026-27.*

PROJECT GUIDE
Ms. Anju John
Assistant Professor
Department of Computer Applications

PROJECT COORDINATOR
Ms. Athira R Kurup
Assistant Professor
Department of Computer Applications

HEAD OF THE DEPARTMENT
Ms. Divya S B
HOD,Assistant Professor
Department of Computer Applications

INTERNAL EXAMINER

ACKNOWLEDGEMENT

I am greatly indebted to the authorities of Mangalam College of Engineering for providing the necessary facilities to successfully complete my Project on the topic "**Accident Detection and Alert System Using Smartphone Sensors**".

I express my sincere thanks to **Dr. Vinodh P Vijayan**, Principal, Mangalam College of Engineering, for providing the facilities and academic environment required to complete my Project successfully.

I thank and express my sincere gratitude to **Ms. Divya S B**, HOD, Assistant Professor, Department of Computer Applications, Mangalam College of Engineering, for her valuable support and encouragement, which helped me in successfully completing this Project work.

I express my gratitude to my Internal Guide, **Ms. Anju John**, Assistant Professor, Department of Computer Applications, for her suggestions, technical guidance and encouragement, which helped in the successful completion of my Project.

Furthermore, I would like to acknowledge with much appreciation the crucial role of the faculties, especially the Project Coordinator, **Ms. Athira R Kurup**,Assistant Professor,Department of Computer Applications, Mangalam College of Engineering, who gave me the necessary guidance, resources and coordination required to complete the project presentation and report.

Finally, I would like to express my heartfelt thanks to my parents, friends and well-wishers for their continuous support, encouragement and motivation throughout the completion of this Project.

ABHISHIKTH S SIBY

MLM25MCA-2004

ABSTRACT

The first minutes after a serious road accident are critical, yet a victim who is injured, unconscious or trapped may be unable to call for help, and this delays medical assistance. Accident Detection and Alert System Using Smartphone Sensors is a Flutter-based mobile safety application designed to overcome this problem by automatically identifying accident-like movements and providing rapid emergency notification. The system uses the accelerometer and gyroscope available in a smartphone to monitor motion continuously when detection mode is enabled. Abnormal acceleration or sudden high-impact movement is evaluated against predefined threshold conditions. When a possible accident is detected, the application activates an audible warning and provides a short user-confirmation window so that a false alert can be cancelled.

If the user does not cancel the warning, the application obtains the current geographical position using the smartphone GPS through the Geolocator service. An emergency message containing the user name, blood group and a Google Maps location link is prepared for a pre-saved emergency contact. The project uses local storage through SharedPreferences for essential user details and settings, reducing dependence on an external database. Its core detection, GPS and SMS workflow is designed to remain useful even where mobile data connectivity is limited, because sensing and message preparation are performed on the device.

The proposed system addresses the delay that can occur when an injured person is unable to manually call for assistance. It provides a low-cost implementation because it uses sensors already present in modern smartphones and does not require dedicated vehicle hardware. The project demonstrates the practical integration of mobile sensing, location services, local storage, alarms and emergency communication in a single application.

Testing covers user-data validation, detection enable/disable behaviour, normal and abnormal movement, false-alert cancellation, countdown handling, location accuracy and emergency-message generation. Future improvements may include machine-learning-based classification, multiple emergency contacts, direct emergency-service integration, wearable support, health monitoring and adaptive sensitivity.

Keywords: *Accident Detection, Smartphone Sensors, Accelerometer, Gyroscope, GPS, SMS Alert, Flutter, Dart, Emergency Response, Mobile Safety.*

Mapping with Sustainable Development Goals (Mention the Goal)

SDG 3 Good Health and Well-Being
SDG 9 Industry, Innovation and Infrastructure
SDG 11 Sustainable Cities and Communities

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	**i**
	ABSTRACT	**ii**
	List of Figures	**iv**
	List of Tables	**v**
	List of Abbreviations	**vi**
Chapter 1	**INTRODUCTION**	**1**
	1.1 Background	1
	1.2 Motivation	2
	1.3 Scope & Objectives	3
	1.4 Problem Statement	4
	1.5 Organisation of the Report	4
Chapter 2	**REQUIREMENTS AND ANALYSIS**	**5**
	2.1 Existing System & Problem Statement	5
	2.2 Solution Overview	5
	2.3 Hardware Requirements	7
	2.4 Software Requirements	7
	2.5 Functional and Non-Functional Requirements	9
	2.6 Feasibility Study	10
	2.7 Permissions and Privacy Considerations	11
Chapter 3	**LITERATURE REVIEW**	**12**
	3.1 Overview	12
	3.2 IoT-Based Smart Accident Detection and Early Warning System for Emergency Response and Risk Management (2025)	12
	3.3 Vehicle Accident Detection and Locating Using GSM and GPS (2024)	13
	3.4 Smart Accident Detection and Alert System (2021)	13
	3.5 A Survey on IoT-Based Automatic Road Accident Detection (2021)	14
	3.6 Prototype Proposal for Quick Accident Detection and Response System (2020)	14
	3.7 A Novel Internet of Things-Enabled Accident Detection and Reporting System for Smart City Environments (2019)	15

CHAPTER NO.	TITLE	PAGE NO.
	3.8 Automatic Accident Detection with Multi-Modal Alert System Implementation for ITS (2016)	15
	3.9 Accident Reporting and Guidance System: With Automatic Detection of the Accident (2016)	16
	3.10 Car Crash Detection on Smartphones (2015)	16
	3.11 iBump: Smartphone Application to Detect Car Accidents (2015)	17
	3.12 WreckWatch: Automatic Traffic Accident Detection and Notification with Smartphones (2011)	17
	3.13 Summary of the Literature Survey	18
	3.14 Research Gap Identified	19
Chapter 4	**DIAGRAMS**	**20**
	4.1 Use Case Diagram	20
	4.2 Class Diagram	21
	4.3 State Diagram	22
	4.4 Activity Diagram	22
	4.5 Sequence Diagram	24
Chapter 5	**METHODOLOGY**	**25**
	5.1 Module Descriptions	25
	5.2 Algorithms	32
	5.3 Architecture	36
	5.4 Case Study	37
Chapter 6	**IMPLEMENTATION DETAILS**	**39**
	6.1 Implementation Procedures	39
	6.2 Testing Methods and Results	40
	6.3 Test Cases	42
	6.4 Implementation Consistency and SMS Mechanism Note	43
Chapter 7	**ADVANTAGES AND DISADVANTAGES**	**45**
	7.1 Advantages	45
	7.2 Disadvantages	45
Chapter 8	**RESULT AND DISCUSSION**	**47**
	8.1 Functional Results	47
	8.2 Discussion of Detection Behaviour	47
	8.3 Discussion of Emergency Communication	48
	8.4 Overall Analysis	48

CHAPTER NO.	TITLE	PAGE NO.
	8.5 Limitations of the Evaluation	49
Chapter 9	**CONCLUSION**	**50**
	9.1 Summary	50
	9.2 Limitations	50
	9.3 Future Scope	51
	9.4 ApplicationsS	53
Chapter 10	**APPENDICES**	**55**
	10.1 Sample Code	55
Chapter 11	**REFERENCES**	**61**

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	Use Case Diagram of Accident Detection and Alert System	22
4.2	Class Diagram of Accident Detection and Alert System	23
4.3	State Diagram of Accident Detection and Alert System	24
4.4	Activity Diagram of Accident Detection and Alert System	25
4.5	Sequence Diagram of Accident Detection and Alert System	26
5.1	Emergency Details Screen	28
5.2	Main Accident Detection	29
5.3	Live Location	32
5.4	Emergency Message Notification	33
5.5	System Architecture of Accident Detection and Alert System	39
10.1	Source Code of the User Details Page	57
10.2	Source Code of the Motion Detector Class	58
10.3	Source Code of the Alert Page State and Notification Set-up	59
10.4	Source Code of Sensor Subscriptions and Motion Evaluation	60
10.5	Source Code of Alert Cancellation and the Start of Escalation	61

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Implemented Features and Proposed Enhancements	6
2.2	Hardware Requirements	7
2.3	Software Requirements	8
2.4	Packages Used and Their Roles	9
2.5	Major Functional Requirements	9
2.6	Non-Functional Requirements	10
3.1	Comparative Summary of Literature	20
4.1	Use Cases of the System	23
5.1	Modules of the Accident Detection and Alert System	27
5.2	Prototype Detection Parameters	36
5.3	Detection Parameters Visible in the Source Snapshot	38
6.1	User Details Page Test Cases	43
6.2	Accident Detection Test Cases	44
6.3	SMS and Location Test Cases	44
6.4	Proposed Additional Tests (Not Executed)	45
6.5	Documented Values Compared with the Source Snapshot	46
8.1	Functional Result Summary	49

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
API	Application Programming Interface
GPS	Global Positioning System
GSM	Global System for Mobile Communications
IMU	Inertial Measurement Unit
ITS	Intelligent Transportation System
MCA	Master of Computer Applications
MVT	Model-View-Template
SDK	Software Development Kit
SMS	Short Message Service
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator

CHAPTER 1
INTRODUCTION

1.1 Background

Road transportation provides essential mobility, but road accidents remain a major safety challenge because the first few minutes after a serious collision are critical. In many situations the victim may be unconscious, trapped or otherwise unable to communicate. A delay between the accident and the delivery of an alert can therefore delay medical assistance. Automatic detection systems aim to reduce this reporting delay by sensing abnormal motion and immediately generating an emergency notification.

Road safety is also a public-health concern. The World Health Organization treats road traffic injuries as a major global health issue and publishes regular status reports on road safety (World Health Organization, 2023). Research on automatic crash notification therefore concentrates on shortening the interval between the impact and the first alert, because this interval depends on whether a capable person is present to raise the alarm rather than on the technology that is available.

Modern smartphones contain sensors that can support such a system without additional dedicated vehicle hardware. An accelerometer measures linear acceleration along the three axes of the phone, while a gyroscope measures rotational motion and orientation changes. GPS provides latitude and longitude, and mobile communication facilities can be used to prepare or transmit an emergency message. These capabilities make the smartphone a practical platform for a low-cost accident detection application.

The project described in this report is a Flutter-based Accident Detection and Alert System. The application stores the user name, blood group and emergency contact number locally. When the user enables detection mode, accelerometer and gyroscope data are monitored continuously. A sudden movement above the configured threshold is treated as a possible accident. The system then produces a warning alarm and allows the user to cancel the event if it is a false detection. If the warning is not cancelled, the current GPS coordinates are obtained and included in an emergency message containing a Google Maps link.

The original project implementation uses Flutter and Dart together with packages such as sensors_plus, geolocator, SharedPreferences, url_launcher, audioplayers and permission_handler. This combination supports sensor monitoring, location acquisition, local data persistence, alert sound generation, permission handling and emergency-message

preparation. The design intentionally keeps the core logic on the mobile device so that it can remain useful in locations where cloud connectivity is unreliable.

1.1.1 Smartphone Sensing Capabilities Used in the Project

The application relies on four capabilities that are present in most Android smartphones. The **accelerometer** reports acceleration along the X, Y and Z axes of the device, and because the reading includes the effect of gravity, a phone lying still reports a magnitude of about 9.8 m/s². The **gyroscope** reports angular velocity about the same three axes and therefore indicates how quickly the phone is rotating, which helps to separate a tumbling event from a straight, hard stop. The **GPS receiver** supplies latitude and longitude through the operating-system location service, and the **cellular messaging facility** allows a short text message to be sent without relying on mobile data.

Each capability has well-known limits. Sensor readings differ between handset models, GPS fixes can be delayed indoors or in tunnels, and messaging depends on cellular coverage and operating-system permissions. These limits are treated as design inputs throughout the report and are summarised in Chapters 7 and 9.

1.2 Motivation

The main motivation for this project is the need to reduce the time required to communicate an emergency after an accident. Traditional reporting depends on the victim or a bystander noticing the event, knowing the exact location, and contacting the correct person or service. This sequence may fail when the victim is unconscious, when the road is isolated, or when witnesses are not immediately available.

A smartphone-based solution is attractive because many users already carry a sensor-rich mobile device during travel. Reusing existing hardware avoids the cost and maintenance burden of installing a separate accident detector in every vehicle. It also makes the concept suitable for drivers, two-wheeler riders, cyclists and people working in environments where sudden falls or impacts may occur.

Another motivation is the possibility of combining detection and communication within one simple user workflow. The app does not only detect a sudden movement; it also stores essential personal information, provides an alarm before escalation, gets the location and prepares an emergency message. The false-alert cancellation stage is especially important because ordinary phone drops, hard braking or rough road conditions can otherwise cause unnecessary alerts.

A further motivation is accessibility. Systems that depend on vehicle-mounted hardware are tied to particular vehicles and do not help pedestrians, cyclists or riders of older vehicles. An application that travels with the person rather than with the vehicle can protect the person in whichever mode of transport is used, subject to the limitations discussed in Chapter 7.

The project is also motivated by its educational value. It demonstrates how mobile application development, real-time sensor streams, threshold algorithms, asynchronous programming, permission management, GPS, local storage and device services can be integrated into one practical safety-oriented system.

1.3 Scope & Objectives

The scope of the project is limited to a smartphone-based accident detection and emergency alert application. It is not intended to replace certified automotive crash-detection hardware or professional emergency dispatch infrastructure. Instead, it provides an accessible supplementary safety mechanism built around sensors already available in the phone.

The objectives of the project are as follows.

- To collect and store the user name, blood group and emergency contact number locally on the device.
- To provide a simple switch for enabling and disabling accident monitoring.
- To monitor accelerometer and gyroscope readings during active detection mode.
- To calculate motion magnitude and compare it with threshold conditions to identify abnormal movement.
- To generate a warning alarm when a possible accident is detected and provide a cancellation opportunity for false alerts.
- To obtain the current GPS location using the Geolocator service after a confirmed event.
- To create an emergency message containing personal details and a Google Maps link to the current coordinates.
- To preserve the most important configuration data locally using SharedPreferences.
- To validate the application through functional test cases covering user input, sensor monitoring, alerts, location and message generation.
- To provide a design that can later be extended with AI/ML, wearable devices, health sensing and direct emergency-service integration.

The expected outcome is a working Android-oriented Flutter application capable of monitoring device movement, warning the user, identifying the current location and generating an emergency SMS workflow with minimal manual intervention.

1.4 Problem Statement

Accident victims may be unable to manually call for help immediately after a serious collision. Existing manual reporting mechanisms can therefore introduce a delay between the accident and the first emergency notification. Some automatic systems require dedicated vehicle hardware, a permanent server connection or expensive infrastructure, reducing their accessibility.

The project addresses the following problem: how can a commonly available smartphone be used to detect accident-like motion and quickly provide a location-aware emergency alert while minimizing false notifications and avoiding dependence on additional hardware?

The proposed answer is to combine accelerometer and gyroscope monitoring with a threshold-based detection mechanism, a user-confirmation countdown, GPS location retrieval and SMS-based communication. The solution must remain easy to configure and should store only the information required for the emergency workflow. It should also allow the user to disable detection when it is not required.

The core technical challenge is balancing sensitivity and reliability. A threshold that is too low can create false positives from normal movement, while one that is too high can miss genuine events. Phone placement, device sensor differences and road conditions can also affect readings. These limitations are acknowledged and form part of the future scope of the project.

1.5 Organisation of the Report

The report is organised into eleven chapters, which follow the order of the college template: introduction, requirements and analysis, literature review, diagrams, methodology, implementation details, advantages and disadvantages, results and discussion, conclusion, appendices and references.

CHAPTER 2
REQUIREMENTS AND ANALYSIS

2.1 Existing System & Problem Statement

In many conventional accident-reporting situations, the process begins only after a victim or witness makes a phone call. The caller must explain that an accident occurred, identify the location and contact relatives, police or medical responders. If the victim is unable to use the phone, the process is entirely dependent on other people at the scene. This critical delay introduces extreme danger during the "Golden Hour" of emergency response, where every minute lost directly impacts the victim's chances of survival.

Some existing technical solutions provide automatic crash notification but rely on vehicle-mounted hardware, OBD interfaces, cloud services or continuous internet connectivity. Such systems can be effective, but they increase deployment cost and may not be available in older vehicles or to users of two-wheelers. Furthermore, cloud-dependent applications often fail completely in remote geographic transit zones where active internet connections drop, rendering them useless precisely when an emergency strikes. Smartphone applications reduce the hardware requirement but can still suffer from false positives caused by phone drops, sudden hand movements or poor threshold calibration.

The problem therefore has both functional and operational dimensions. The system must detect unusual movement, distinguish it from normal use sufficiently well for a mini-project prototype, notify the user before escalation, retrieve the current location and prepare an emergency message. At the same time it must remain lightweight, understandable and affordable.

2.2 Solution Overview

The proposed Accident Detection App is developed using Flutter. The user initially enters personal details including name, blood group and emergency contact number. These values are stored locally using SharedPreferences. The main screen includes an Accident Detection switch. When the switch is enabled, the application subscribes to accelerometer events and evaluates the magnitude of device acceleration in real time.

The implementation maintains a small sliding window of recent magnitudes and evaluates both an absolute threshold and a spike relative to the average. This allows the app to react to a large acceleration value or a sudden change compared with the immediately preceding

motion. A cooldown interval prevents the same physical event from generating many repeated detections.

When the detection condition is met, the app plays an alarm. The documented project workflow provides a short user-response period to cancel a false alert. If the event is treated as a real accident, the Geolocator service is used to obtain the current latitude and longitude. A Google Maps URL is constructed and inserted into an emergency message along with the stored user details. The message is addressed to the saved emergency number using the phone SMS mechanism.

This architecture has no central application database. Personal information and settings are stored locally. By utilizing Flutter's shared_preferences plugin, the system writes the user's name, blood group, and emergency contact details directly into the device's persistent XML or property-list configuration layer. This decentralized storage strategy completely removes the privacy hazards, data breach vectors, and server overhead associated with hosting user credentials on a remote cloud backend. Furthermore, the application executes its core sensor stream evaluation and message composition workflows asynchronously within the on-device memory, ensuring immediate functionality even in remote transit zones with limited data network signals. The design reduces infrastructure requirements and supports a straightforward emergency workflow. It also creates a clear foundation for future enhancements such as adaptive thresholds and multiple emergency recipients.

The prototype can be described in terms of what has been implemented and what is proposed only as future work. Table 2.1 keeps the two apart so that the report does not claim features that are not present in the application.

Table 2.1: Implemented Features and Proposed Enhancements

Feature	Status	Remarks
User details (name, blood group, emergency number) with validation and local storage	Implemented	Shown in Figure 5.1; stored with SharedPreferences
Detection switch (Start / Stop Detection)	Implemented	Shown in Figure 5.2
Accelerometer and gyroscope monitoring with magnitude calculation	Implemented	Visible in the source screenshots of Chapter 10
Cooldown to avoid repeated alerts for one event	Implemented	8000 ms in the documented prototype and in the source snapshot
Looping warning alarm and 20-second cancellable countdown	Implemented	Described in the on-screen note of Figure 5.2 and in the source snapshot
GPS retrieval with location-service and permission checks	Implemented	Geolocator calls visible in Figure 10.5
Google Maps link inside the emergency message	Implemented	Message text visible in Figure 5.4

Feature	Status	Remarks
Emergency text message to one saved contact	Implemented	Mechanism discussed in Section 6.4
Machine-learning classification, adaptive thresholds	Proposed	Future scope, Section 9.3
Multiple contacts, wearable data, OBD-II data, emergency-service APIs, cloud history	Proposed	Future scope, Section 9.3

2.3 Hardware Requirements

Table 2.2: Hardware Requirements

Component	Specification / Requirement
Development Computer	Windows 10 or newer, macOS 10.14 or later, or Linux/Ubuntu
Processor	Intel Core i3 or higher, approximately 2.0 GHz or above
RAM	Minimum 4 GB; 8 GB recommended for comfortable Flutter/Android development
Storage	At least 64 GB free space for SDKs, project files and emulator images
Network	Required for SDK/package installation and documentation; not required for core on-device sensing workflow
Mobile Device	Android smartphone with accelerometer, gyroscope and GPS/location capability
Audio	Device speaker for warning alarm
SIM / Messaging	SMS-capable device or SMS composer access for emergency communication

A physical Android device is preferred for realistic sensor testing because virtual emulators rely on static sliders that fail to mimic the dynamic, complex micro-vibrations and multi-axis forces experienced during actual vehicle transit. A real smartphone feeds authentic raw data from its internal Inertial Measurement Unit (IMU) straight to the system kernel, allowing developers to observe how real road defects like potholes affect sliding window averages. Furthermore, physical testing is necessary to programmatically manage real-world operating system behaviors, such as aggressive low-power background execution policies or deep-sleep sensor stream throttling, that emulators bypass. Consequently, GPS, SMS behaviors, and runtime permissions are best validated on a real device to ensure system reliability.

2.4 Software Requirements

Table 2.3: Software Requirements

Software Component	Specification
IDE	Android Studio or Visual Studio Code
Framework	Flutter SDK 3.x or compatible
Programming Language	Dart
Sensor Package	sensors_plus
Location Package	geolocator
Local Storage	shared_preferences
SMS / URL Handling	url_launcher or supported SMS package
Audio	audioplayers
Permissions	permission_handler
Build Tools	Android SDK and Gradle
Version Control	Git / GitHub (optional for project management)

The project source identifies these packages as the main software dependencies. Flutter provides the cross-platform user-interface framework, while Dart handles asynchronous event streams and device-service calls. The implementation integrates key plugins such as sensors_plus for stream-based accelerometer parsing and geolocator for background location retrieval. These tools allow the software architecture to remain lightweight and fully modular without requiring external database wrappers. Android permissions must be declared for location and messaging features.

2.4.1 Flutter and Dart

Flutter is a user-interface toolkit that builds applications from a single Dart code base. The screens of the project are composed of widgets, and the source snapshot uses stateful widgets (for example `UserDetailsPage` and the alert-page state class) so that the interface is redrawn when stored data, the detection switch or the countdown changes. Dart is the language used by Flutter. It provides `Future` and `async/await` for operations that complete later, such as reading SharedPreferences or requesting a GPS position, and `Stream` objects for continuous data such as sensor events.

These features suit the project because accident detection is event driven. The application does not poll the sensors in a loop; it subscribes to the sensor streams and reacts each time a new reading arrives, using `Timer` objects for the warning beep and for the countdown and a `ValueNotifier` to update the countdown value shown to the user.

2.4.2 Packages Used and Their Roles

The packages listed in Table 2.4 are the dependencies identified in the project source. Each package is responsible for one device capability, which keeps the application code modular.

Table 2.4: Packages Used and Their Roles

Package	Role in the application
sensors_plus	Provides accelerometer and gyroscope event streams
geolocator	Checks whether location services are enabled, checks and requests location permission, and returns the current position
shared_preferences	Stores the user name, blood group, emergency number and detection state as key-value pairs on the device
audioplayers	Plays the looping warning sound through an audio player object
permission_handler	Handles runtime permission requests for location and messaging
url_launcher	Launches the SMS URI that carries the recipient and message body
flutter_local_notifications	Initialised in the alert page of the source snapshot (Figure 10.3); not listed in the original software table

2.5 Functional and Non-Functional Requirements

Functional Requirements

Table 2.5: Major Functional Requirements

ID	Functional Requirement	Implemented by
FR1	The system shall allow the user to enter and save name, blood group and emergency contact number.	User Details
FR2	The system shall validate that required user-detail fields are not empty.	User Details
FR3	The user shall be able to enable or disable accident detection.	Detection Control
FR4	The application shall read accelerometer and gyroscope/motion data while detection is enabled.	Sensor Monitoring
FR5	The detection logic shall identify movements exceeding configured conditions.	Sensor Monitoring
FR6	The system shall produce a warning alarm for a suspected accident.	Alert Generation
FR7	The user shall be able to cancel a false alert during the warning period.	Alert Generation
FR8	The system shall obtain latitude and longitude when escalation is required.	Location Tracking
FR9	The system shall create an emergency message containing user details and a Google Maps location link.	Emergency Message
FR10	The system shall avoid repeatedly sending multiple alerts for a single detected event.	Sensor Monitoring

The last column of Table 2.5 links every requirement to the module of Section 5.1 that implements it, which makes the requirements traceable to the design.

Non-Functional Requirements

Table 2.6: Non-Functional Requirements

ID	Category	Requirement	How it is checked
NFR1	Usability	The interface should be simple enough to configure without technical knowledge.	Usability testing (Section 6.2)
NFR2	Performance	Sensor evaluation should occur in real time with minimal delay.	Observed during functional tests; not measured numerically
NFR3	Reliability	Saved user details should persist between application launches.	Persistence test, Table 6.1 (case 4)
NFR4	Privacy	Only essential personal details should be stored locally and not unnecessarily transmitted.	Design review (Section 2.7)
NFR5	Maintainability	Sensor thresholds and package integrations should be isolated enough to permit future updates.	Code structure (Chapter 10)
NFR6	Portability	The Flutter code base should remain compatible with commonly used Android devices, subject to permission and hardware support.	Device and compatibility testing (Section 6.2)

2.6 Feasibility Study

The feasibility study examines whether the proposed Accident Detection and Alert System can be developed and used successfully with the available technologies, resources and time. The system relies on sensors, GPS, audio and messaging features already present in a smartphone, and the economic, technical, operational and schedule aspects are discussed below.

2.6.1 Economic Feasibility

The system is economically feasible because it uses the smartphone's existing sensors, GPS, speaker and messaging features. It does not require an external microcontroller, dedicated crash sensor, separate GPS receiver or paid backend server for the core prototype. By processing telemetry data entirely on-device, the application circumvents the heavy infrastructure expenses associated with cloud computing networks and remote data servers. Furthermore, the complete exclusion of specialized automotive hardware retrofits removes physical deployment overhead for everyday commuters. This zero-hardware reliance ensures the safety utility remains completely free of manufacturing and maintenance liabilities. Development tools and the Flutter framework are available without licensing cost for academic use.

2.6.2 Technical Feasibility

The project is technically feasible with current Android smartphones and the Flutter ecosystem. Packages are available for motion sensors, location, local preferences, audio

and URL/SMS launching. The open-source community provides mature, field-tested plugins like sensors_plus and geolocator that grant direct asynchronous access to underlying hardware layers. Additionally, Dart's built-in stream handling allows the application to capture and compute real-time acceleration vectors natively with minimal memory overhead. Modern IDEs like Android Studio and Visual Studio Code also simplify cross-compilation and automated build pipelines using Gradle. The main technical risks are sensor variation between devices, background execution restrictions, permission handling and the tuning of thresholds to reduce false detections.

2.6.3 Operational Feasibility

The operational workflow requires only an initial setup followed by a simple detection toggle. The user does not need to interpret sensor values. When a potential accident is identified, a clear warning allows cancellation. Upon application initialization, a straightforward configuration panel ensures that medical identifiers and emergency contacts are securely written to device memory in less than a minute. Once active tracking is enabled, the interface dynamically transitions to a minimalist state to eliminate visual clutter and avoid driver distraction during transit. The inclusion of high-contrast UI controls and explicit auditory countdown indicators ensures the system remains fully usable even under high-stress conditions. This makes the solution practical for drivers and riders who need a low-interaction safety tool.

2.6.4 Schedule Feasibility

The system is modular enough to be developed in stages: user details and storage, sensor monitoring, detection logic, alarm handling, location, SMS integration, testing and documentation. By dividing the development lifecycle into self-contained milestones, each feature can be independently coded and verified without impacting the rest of the application. For example, the user registration interface and SharedPreferences database layer can be fully completed and debugged before initiating any complex background sensor logic. This structural isolation prevents late-stage integration bottlenecks and allows for a smooth, predictable path toward assembly.

2.7 Permissions and Privacy Considerations

The application needs access to the device location and to messaging, and both are protected by Android permissions. Android requires such permissions to be declared in the manifest (which was not part of the supplied screenshots), and the application checks and requests them at run time through the Geolocator and permission_handler packages.

Privacy is handled by keeping the stored data small: SharedPreferences holds plain key-value pairs in the private storage of the application and is not an encrypted store, which is why only the details needed for the emergency message are kept.

CHAPTER 3
LITERATURE REVIEW

3.1 Overview

Research on automatic accident detection has evolved from vehicle-mounted sensors toward smartphone sensing, connected vehicles and IoT platforms. The literature shows several recurring design ideas: detecting high acceleration, combining multiple sensors to reduce false positives, using GPS to identify the accident site, providing a short cancellation period, and transmitting information to emergency contacts or services. The eleven works reviewed below were selected because they relate directly to smartphone sensing, accident detection, emergency notification or intelligent transportation systems.

The works can be read as a timeline. WreckWatch (2011) showed that an ordinary smartphone could detect and report an accident. In 2015 and 2016 the idea was developed in several directions: iBump applied pattern-recognition methods to accelerometer data, Lahn et al. described a simple Android crash-detection algorithm, Fernandes et al. combined smartphone sensors with vehicle data, and Fanca et al. built a reporting and guidance system around GPS. Bhatti et al. (2019) and the survey and prototype papers of 2020 and 2021 moved towards IoT platforms, while the 2024 and 2025 works combine GSM, GPS, smartphone sensors and early-warning services. The present project follows the smartphone-centred line of this development and deliberately keeps the processing simple and on the device.

Each work is summarised below from the information available in this report, with its method, its main contribution and its relevance to the present project. Where a limitation is stated, it is a limitation that follows from the summary of the work and not an evaluation of the original authors' reults. Unlike approaches that may depend on additional vehicle hardware or complex external infrastructure, the proposed project focuses on using the sensors and communication capabilities already available on a smartphone. Its design prioritises simplicity, affordability, and ease of use. The warning and cancellation mechanism is intended to help reduce false emergency alerts, while GPS-based location information can help the emergency contact identify the possible accident location. SMS-based notification can also be useful when mobile internet access is unavailable, although successful delivery still depends on cellular network availability and the required phone permissions.

3.2 IoT-Based Accident Detection Using Smart Helmet (2025)

Sheeba Rachel, Pragadesh H, and Yuvaraj R (2025) proposed an IoT-based accident detection system using a smart helmet to improve motorcycle rider safety and provide faster emergency assistance. The main objective of the system is to reduce the time required to identify road accidents and inform emergency contacts. The proposed system integrates Internet of Things (IoT) technology with different sensors to continuously monitor the rider's condition and identify possible accidents. The major components include an accelerometer, GPS module, alcohol sensor, and infrared (IR) sensor. The accelerometer detects sudden movements, impacts, and abnormal changes in the helmet's position, while the GPS module identifies the rider's current geographical location. The alcohol sensor helps detect alcohol consumption, and the IR sensor can be used to monitor helmet-related conditions, depending on the system configuration.

When a possible accident is detected, the system initiates an alert process to notify emergency contacts and nearby hospitals through a connected mobile application. The GPS module provides location information, allowing emergency responders to identify the accident site and reach the injured rider more quickly. One important feature of the system is a one-minute confirmation period, which gives the rider time to respond before the emergency alert is sent. This feature helps reduce false alerts caused by sudden braking, minor impacts, or other normal movements that may be incorrectly identified as accidents. If the rider does not respond within the specified period, the system proceeds with the emergency notification process. By combining sensor-based monitoring, GPS tracking, IoT connectivity, and mobile communication, the proposed system aims to improve accident detection and emergency response. However, its effectiveness depends on sensor accuracy, proper helmet usage, network availability, and reliable communication with emergency contacts. The study demonstrates the potential of IoT-based smart helmets to improve motorcycle safety and reduce delays in providing emergency assistance.

3.3 Vehicle Accident Detection and Locating Using GSM and GPS (2024)

This conference paper (International Conference on Communication, Computing and Internet of Things, 2024) belongs to the group of systems that detect an accident from motion data and then use GPS positioning and GSM communication to report where it occurred. In the comparative summary of this report its primary method is recorded as accelerometer, GPS and GSM and its relevance as accident-location SMS. Detailed

experimental results of the paper are not reproduced here because they are not part of the material supplied for this report.

Limitations and relevance: The paper supports the choice of text messaging as a channel that does not need mobile data. GSM/GPS systems of this kind are usually built around separate modules, whereas the present project uses the equivalent facilities that already exist in a smartphone.

3.4 Smart Accident Detection and Alert System (2021)

This paper presents a smart accident detection and alert system designed to automatically notify emergency contacts when a vehicle accident occurs. The system detects an accident using vehicle sensors and sends an SMS containing the detected location to registered emergency contacts. A reset button is also provided so that the alert can be cancelled when the people inside the vehicle are safe, helping to reduce unnecessary emergency notifications. The main purpose of the system is to reduce the delay between an accident and the notification of emergency contacts. By automatically detecting the accident and sharing the location through SMS, the system can provide important information without requiring the user to manually make a call or send a message. This work is particularly relevant to the proposed system because it also considers automatic detection, location sharing, SMS notification, and false-alert cancellation. However, the proposed project extends this idea by implementing the complete functionality as a smartphone application using built-in smartphone sensors and a warning period before sending the final alert.

Limitations and relevance: Detection relies on vehicle sensors and cancellation is performed with a reset button, so the system is tied to the vehicle. The present project transfers the same cancellation idea to an on-screen control inside a smartphone application.

3.5 A Survey on IoT-Based Automatic Road Accident Detection (2021)

This paper provides a detailed survey of different technologies used for automatic road accident detection and information sharing. It discusses several approaches, including smartphone-based accident detection, GSM, GPS, vehicular ad hoc networks, machine-learning algorithms, and mobile applications. The study emphasizes that rapid accident identification and communication of the accident location are important for improving emergency response. The survey also examines the advantages, limitations, and challenges associated with existing accident detection methods. It shows that different systems use different combinations of sensors and communication technologies depending on their requirements. The study is useful for identifying the major technologies and research

directions in automatic accident detection. For the proposed project, this survey provides a foundation for selecting smartphone sensors, GPS, and SMS as the major components of the accident detection and alert mechanism.

Limitations and relevance: As a survey it does not provide an implementation or results of its own. It is used in this report as background for selecting smartphone sensors, GPS and SMS as the main components.

3.6 Prototype Proposal for Quick Accident Detection and Response System (2020)

Rana et al. proposed a Quick Accident Response System (QARS) using IoT technology to support faster emergency assistance after road accidents. The system uses accelerometer and ultrasonic/proximity sensors to identify possible accidents. When an accident is detected, the system obtains the exact accident location using a GPS-GSM module and sends information to nearby Emergency Response Units through a dedicated mobile application. The proposed system also supports video recording and sharing of accident information, while an offline feature allows the accident location and alert to be sent through a simple text message to emergency contacts or emergency response units. This makes the system useful in situations where Internet connectivity may not be available. The work demonstrates the importance of combining automatic sensing, location detection, communication, and emergency response. The proposed project follows a similar idea but focuses on using the smartphone itself as the main sensing and communication device, reducing the need for additional hardware.

Limitations and relevance: The system needs a GPS-GSM module and ultrasonic or proximity sensors in addition to the mobile application. Its offline text-message feature is the part most relevant to the present project, which also relies on a text message rather than on internet connectivity.

3.7 A Novel Internet of Things-Enabled Accident Detection and Reporting System for Smart City Environments (2019)

Bhatti et al. proposed an IoT-enabled accident detection and reporting system for smart-city environments. The system makes use of smartphone sensor data to determine whether an accident has occurred and automatically report the incident. The researchers focused on reducing false detections and improving the reliability of accident reporting. The system was evaluated using a simulated environment, and the study also considered the role of mobile edge computing in improving latency, security, and privacy. The work demonstrates how smartphones can act as sensing devices within an IoT environment and communicate

accident information to other systems. It also highlights that Internet connectivity and communication infrastructure can affect the performance of such systems. This study is relevant to the proposed project because both approaches use smartphone sensing for accident detection, while the proposed system additionally focuses on direct emergency SMS communication and a user-controlled warning period to handle false alerts.

Limitations and relevance: The evaluation was carried out in a simulated environment, and the reliance on connectivity and infrastructure can affect performance. The attention given to false detections is directly relevant to the user-confirmation stage of the present project.

3.8 Automatic Accident Detection with Multi-Modal Alert System Implementation for ITS (2016)

Fernandes et al. proposed an automatic accident detection system for Intelligent Transportation Systems (ITS) using multiple sources of information. The system combines vehicle information obtained through OBD-II with smartphone sensors such as the accelerometer, magnetometer, and gyroscope. The system supports multiple communication and alert mechanisms for reporting accidents. This approach demonstrates that combining different sensor sources can provide more information for detecting abnormal vehicle conditions. The use of multiple communication methods is also intended to improve the reliability of accident notification. The study is relevant to the proposed system because it demonstrates the usefulness of smartphone motion sensors and automatic emergency alerts. However, the proposed project is designed as a simpler smartphone-based solution that does not require an OBD-II connection or specialized vehicle hardware.

Limitations and relevance: The approach needs an OBD-II connection to the vehicle. It shows the value of combining sensor sources, but also the extra hardware dependency that the present project tries to avoid.

3.9 Accident Reporting and Guidance System: With Automatic Detection of the Accident (2016)

Fanca, Puscasiu, and Valean proposed a smartphone-based accident reporting and guidance system that uses the hardware and software capabilities already available in mobile phones. The system is designed to automatically detect an accident and associate accident information with a geographical location using GPS tracking. It can then create an accident report containing relevant information about the incident. The study shows that smartphones can be effectively used as terminals for accident reporting because they

combine sensors, GPS, processing capabilities, and communication features in a single device. This reduces the requirement for separate accident-reporting equipment. The work is closely related to the proposed project because the proposed system also uses a smartphone for accident detection and location-based emergency reporting. The proposed project further adds a warning and cancellation period and an emergency message that contains a Google Maps link.

Limitations and relevance: The summary stresses reporting and guidance rather than false-alert control. The present project concentrates on the first minutes after an impact, when the user may need to cancel or may be unable to respond.

3.10 Car Crash Detection on Smartphones (2015)

Lahn, Peter, and Braun presented a simple car crash detection algorithm implemented on Android smartphones. The algorithm uses information from the smartphone's accelerometer and location sensor and combines these data sources to identify typical patterns associated with car crash situations. The researchers also presented implementation details and initial evaluation results for the proposed method. The study is important because it demonstrates that dedicated accident-detection hardware is not always necessary when smartphones already contain motion and location sensors. Using the accelerometer allows sudden changes in movement to be identified, while location information can help determine where an incident has occurred. This concept is directly related to the proposed project, which also uses smartphone motion sensors and GPS. However, the proposed system combines these capabilities with a 20-second warning period and an emergency message containing a Google Maps link, providing additional functionality for practical emergency situations.

Limitations and relevance: The paper reports initial evaluation results only, and no accuracy figures are quoted in the supplied summary. It is relevant as evidence that a phone's own accelerometer and location sensor can be sufficient.

3.11 iBump: Smartphone Application to Detect Car Accidents (2015)

Aloul et al. developed iBump, a smartphone application that automatically detects and reports car accidents. The system continuously collects data from the smartphone's accelerometer and analyzes the data using Dynamic Time Warping (DTW) and Hidden Markov Models (HMM). These techniques are used to identify accident patterns, estimate accident severity, and reduce false-positive detections. After an accident is confirmed, the system can send an SMS containing information such as the accident location, severity, and

user information to emergency services and family members. The study reported that the proposed pattern-based approach achieved a low false-positive rate in its testing and was evaluated using simulated crash data. This work is especially relevant because it demonstrates how smartphone sensor data can be processed intelligently instead of depending only on a simple threshold. The proposed project uses a simpler threshold-based detection approach so that it can be implemented efficiently in a lightweight Flutter application without requiring complex machine-learning models.

Limitations and relevance: Pattern-based detection with DTW and HMM needs training data and more processing, and it was evaluated with simulated crash data. The present project accepts a simpler threshold approach and relies on user confirmation instead of a trained model.

3.12 WreckWatch: Automatic Traffic Accident Detection and Notification with Smartphones (2011)

White et al. introduced WreckWatch, an early smartphone-based system for automatic traffic accident detection and notification. The system uses smartphone capabilities, including accelerometer and audio information, to detect possible traffic accidents. The system then uses the available communication and location capabilities to report accident information and support emergency response. The major motivation of the work is to reduce the time between an accident and the dispatch of emergency medical personnel. WreckWatch is significant in the development of smartphone-based accident detection because it demonstrated that ordinary smartphones could be used to detect accident-related events without requiring expensive dedicated equipment. The work provides an early foundation for later smartphone-based accident detection systems such as iBump and other IoT-based approaches. The proposed project follows this smartphone-based concept while using modern Flutter development, accelerometer and gyroscope monitoring, GPS location, a 20-second false-alert cancellation period, and SMS-based emergency notification.

Limitations and relevance: As an early system it predates current mobile frameworks, and it uses audio as well as acceleration. It is the conceptual starting point of smartphone-based accident detection.

3.13 Summary of the Literature Survey

The works surveyed above are compared in Table 3.1 on the basis of their primary method and their relevance to the present project, and the research gap is discussed in Section 3.14.

Table 3.1: Comparative Summary of Literature

Paper	Primary Method	Main Relevance
IoT-Based Accident Detection Using Smart Helmet. (2025)	Smartphone sensors + IoT + GPS	sensor-based accident detection, automatic emergency alerts for faster accident response.
Vehicle Accident Detection (2024)	Accelerometer + GPS + GSM	Accident location SMS
Smart Accident Detection (2021)	Sensors + SMS + reset	Alert and false-alert control
IoT Accident Survey (2021)	IoT + GPS/GSM + ML	Review of detection methods
Quick Accident Response (2020)	Accelerometer + GPS/GSM	Automatic emergency response
Bhatti et al. (2019)	Multiple smartphone sensors + IoT cloud	Multi-sensor accident detection for smart cities
Fernandes et al. (2016)	OBD-II + smartphone sensors	Multi-modal accident alerts
Fanca et al. (2016)	Smartphone + GPS	Automatic reporting and guidance
Lahn et al. (2015)	Accelerometer + location sensor	Simple crash detection on smartphones
Aloul et al. (2015)	Accelerometer + DTW/HMM	Detection and false-positive reduction
White et al. (2011)	Accelerometer + GPS + audio	Early smartphone accident detection

"Not stated" means that the feature is not mentioned in the summary of the work available for this report; it does not mean that the original work lacks the feature.

Critical observation: The reviewed studies show that smartphone sensors, GPS and SMS/IoT communication can provide a practical platform for automatic accident detection and emergency reporting. However, the studies also indicate important challenges such as false detections, sensor limitations, phone placement and dependence on additional hardware or network connectivity.

Advanced methods such as DTW/HMM and multi-sensor systems can improve detection, but they may increase system complexity. The present project addresses false alerts through a user confirmation stage and uses a lightweight smartphone-based approach.

3.14 Research Gap Identified

The literature shows that smartphone sensing, GPS positioning, and SMS notification have been successfully used for accident detection and reporting. However, many existing systems use pattern-recognition algorithms, multi-sensor fusion, vehicle data, IoT infrastructure, or cloud processing, which can increase system complexity and deployment requirements. The following research gaps are identified from the surveyed works:

- **Pattern-recognition approaches:** Methods such as iBump improve the distinction between normal and accident events, but require additional data, training, and processing.
- **Multi-modal systems:** Systems such as HDy Copilot use vehicle data and multiple sensors, but may require OBD-II interfaces or additional vehicle hardware, making deployment more complex.
- **IoT and cloud dependency:** IoT-based systems can provide real-time monitoring and reporting, but may depend on Internet connectivity, cloud services, or external infrastructure.
- **False-positive detection:** Simple threshold-based smartphone systems are inexpensive and easy to implement, but may generate false alerts due to phone drops, potholes, sudden movements, or abrupt changes in motion.

The present project therefore adopts a lightweight and smartphone-based approach that uses built-in motion sensors for accident detection. It applies a threshold-based detection method with a sliding-window mechanism and includes a user-confirmation stage to reduce false alerts. Once an accident is confirmed, the system uses GPS to obtain the location and sends an emergency message containing a Google Maps link to the registered contact. User and emergency-contact details are stored locally, avoiding the need for a separate application server. The proposed system provides a low-cost and simple accident detection solution, with scope for future enhancement using adaptive thresholds and machine-learning-based classification.

CHAPTER 4
DIAGRAMS

This chapter presents the UML views of the system. The diagrams are design-level models: they show responsibilities and flow rather than the exact class layout of the source code, which is shown in the appendix screenshots of Chapter 10.

4.1 Use Case Diagram

The use-case view defines the operational boundaries of the system by mapping how the User and Emergency Contact interact with the core mobile application. The primary User remains responsible for initial setup tasks, which include entering personal details, granting device permissions, and toggling active sensor monitoring. If an abnormal movement is detected, the system triggers a warning alarm that allows the User to manually cancel the alert if it is a false alarm. If the countdown expires without user response, the application automatically bypasses manual input to fetch the live GPS coordinates and dispatch a location-aware emergency SMS directly to the Emergency Contact.

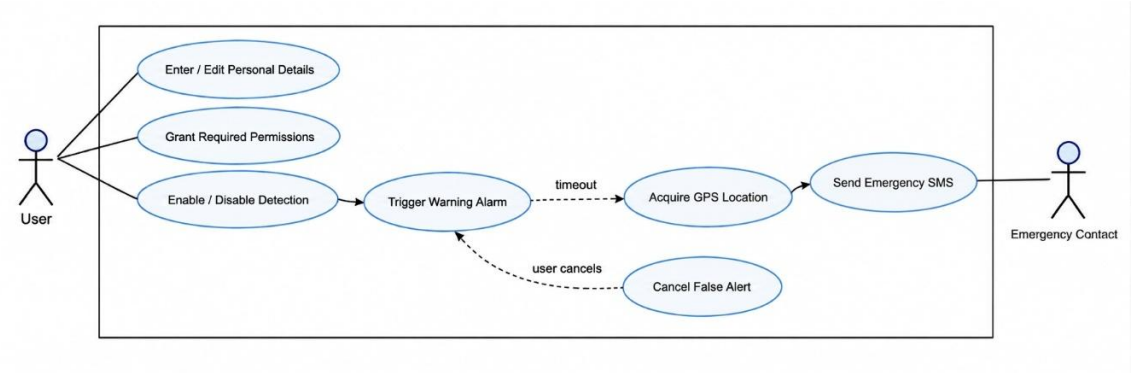


Figure 4.1: Use Case Diagram of Accident Detection and Alert System

The diagram has two actors. The **User** is on the left and is associated with the use cases Enter / Edit Personal Details, Grant Required Permissions and Enable / Disable Detection. The **Emergency Contact** is on the right and is associated only with Send Emergency SMS, because the contact is a recipient and does not operate the application. The rectangle is the system boundary.

Inside the boundary the use cases are connected by the flow of an alert. Enabling detection leads to Trigger Warning Alarm. A dashed arrow labelled timeout leads from the alarm to Acquire GPS Location and then to Send Emergency SMS, while a dashed arrow labelled user cancels connects Cancel False Alert to the alarm, showing that the user can stop the sequence before any location or message is prepared. Table 4.1 describes each use case.

Table 4.1: Use Cases of the System

No.	Use case	Actor	Description
UC1	Enter / Edit Personal Details	User	Enter name, blood group and emergency number; the values are validated and saved
UC2	Grant Required Permissions	User	Allow location and messaging access when requested
UC3	Enable / Disable Detection	User	Start or stop sensor monitoring with the Start / Stop Detection button
UC4	Trigger Warning Alarm	System	Sound the alarm and start the 20-second countdown when abnormal motion is detected
UC5	Cancel False Alert	User	Stop the alarm and countdown so that no message is prepared
UC6	Acquire GPS Location	System	Obtain latitude and longitude when the countdown ends without cancellation
UC7	Send Emergency SMS	System, Emergency Contact	Prepare the message with name, blood group and Maps link for the saved contact

4.2 Class Diagram

The class diagram models the main logical responsibilities of user details, detection control, sensor monitoring, location, alerting and local storage.

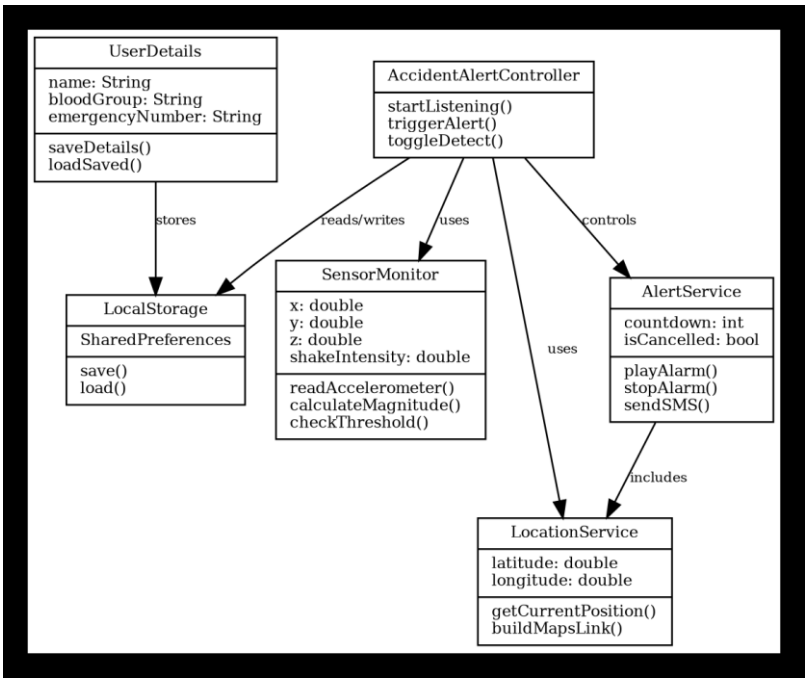


Figure 4.2: Class Diagram of Accident Detection and Alert System

Each box shows the class name, its attributes and its operations. **AccidentAlertController** is the central class: it starts listening to the sensors, triggers an alert and toggles detection, and it is linked to the other classes. It reads and writes **LocalStorage** (SharedPreferences), uses **SensorMonitor** to read the accelerometer, calculate the magnitude and check the

threshold, uses **LocationService** to obtain a position and build the Maps link, and controls **AlertService**, which holds the countdown and cancellation flag and plays the alarm. **UserDetails** stores the three emergency details through LocalStorage, and AlertService includes the location when it sends the message.

In the source snapshot these responsibilities are grouped differently: a motion-detector class (`AccidentMotionDetector`) holds the detection logic, and the alert page state class holds the countdown, alarm, location and message steps. The class diagram therefore shows the logical responsibilities, not the file structure.

4.3 State Diagram

The state diagram represents transitions from idle to monitoring, possible accident, warning, location retrieval and message dispatch.

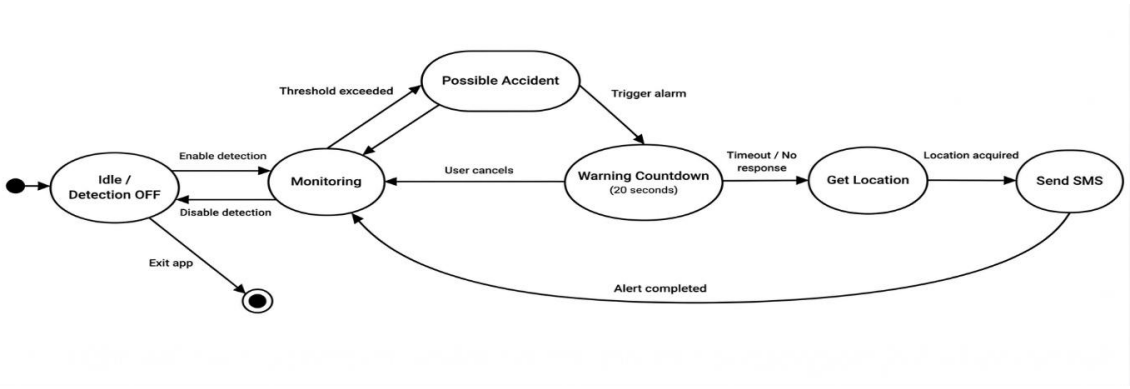


Figure 4.3: State Diagram of Accident Detection and Alert System

The diagram starts at the initial node and enters **Idle / Detection OFF**. The transition Enable detection moves the system to **Monitoring**, and Disable detection returns it to Idle, from where Exit app leads to the final node. From Monitoring, the transition Threshold exceeded leads to **Possible Accident**, and Trigger alarm leads to **Warning Countdown (20 seconds)**. An unlabelled return arrow is also drawn from Possible Accident to Monitoring.

From the countdown state there are two exits. User cancels returns directly to Monitoring, and Timeout / No response leads to **Get Location**, then Location acquired leads to **Send SMS**. After the message step, Alert completed returns the system to Monitoring, so that detection continues without restarting the application.

4.4 Activity Diagram

The activity diagram emphasizes the decision points for threshold detection and user cancellation.

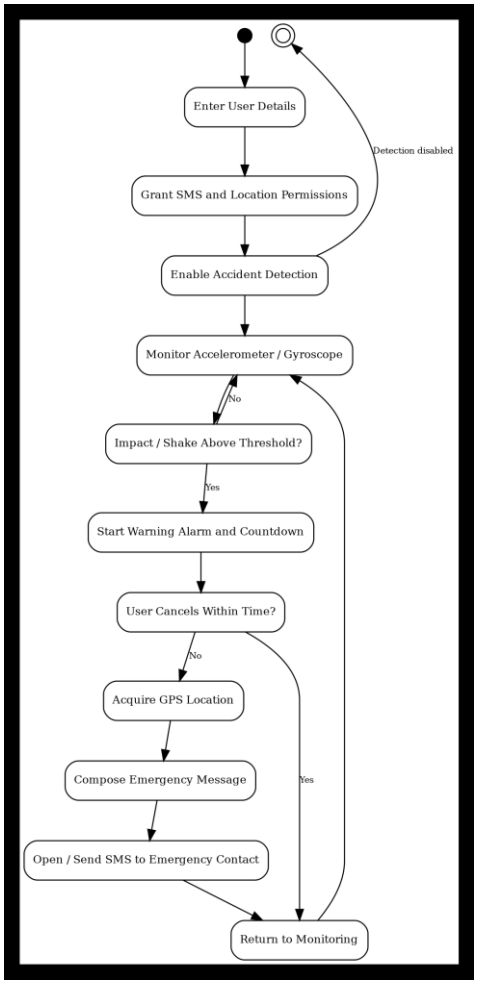


Figure 4.4: Activity Diagram of Accident Detection and Alert System

The diagram reads from top to bottom. It has two decision points: **Impact / Shake Above Threshold?** and **User Cancels Within Time?**. The set-up activities come first, and the monitoring loop and the escalation path follow.

1. Enter User Details, then Grant SMS and Location Permissions.
2. Enable Accident Detection; if detection is disabled, the flow ends at the final node.
3. Monitor Accelerometer / Gyroscope continuously.
4. Decision 1: if the impact or shake is not above the threshold (No), monitoring continues.
5. If it is above the threshold (Yes), Start Warning Alarm and Countdown.
6. Decision 2: if the user cancels within the time (Yes), the flow goes to Return to Monitoring.
7. If the user does not cancel (No), Acquire GPS Location, then Compose Emergency Message.
8. Open / Send SMS to Emergency Contact, then Return to Monitoring.

The activity name Open / Send SMS is deliberately worded to cover both opening the phone's SMS composer and sending the message, since the exact mechanism depends on the build and on platform rules (Section 6.4).

4.5 Sequence Diagram

The sequence diagram presents the time-ordered interactions among the user interface, controller, sensors, location service and emergency messaging service.

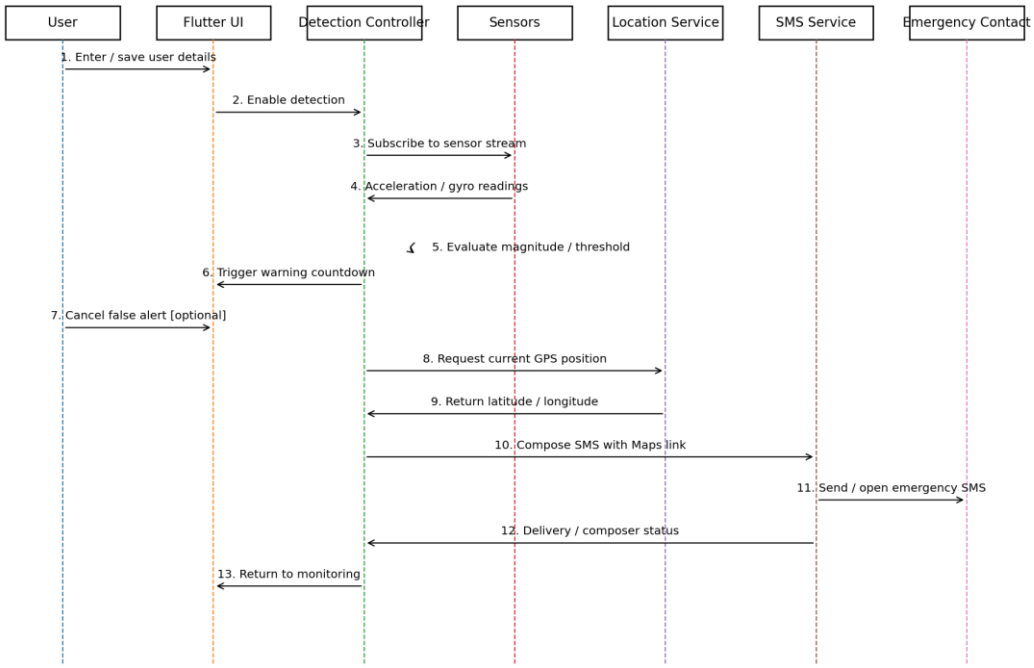


Figure 4.5: Sequence Diagram of Accident Detection and Alert System

Seven participants are shown from left to right: User, Flutter UI, Detection Controller, Sensors, Location Service, SMS Service and Emergency Contact. Messages are numbered 1 to 13 in time order. The detection controller is the hub: it subscribes to the sensor stream, evaluates each reading, calls back to the UI to trigger the warning, and later requests the position and composes the message. Step 7 (cancel false alert) is marked optional; when the user cancels, steps 8 to 12 do not take place.

CHAPTER 5
METHODOLOGY

5.1 Module Descriptions

The Accident Detection and Alert System is divided into seven modules, listed in Table 5.1. The first two modules form the user-facing part of the application, where the user enters details and controls detection. The remaining modules form the system part, which senses motion, raises the alert, finds the location, prepares the message and manages storage and permissions. Each module has a focused responsibility so that sensing, emergency handling and user configuration remain understandable and maintainable.

Table 5.1: Modules of the Accident Detection and Alert System

No.	Module	Responsibility
1	User Details and Settings Module	Collects, validates and stores the user name, blood group and emergency contact number, and reloads them when the page opens.
2	Detection Control Module	Provides the Accident Detection switch, checks that user details exist before enabling detection, stores the detection state and starts accelerometer listening.
3	Sensor Monitoring Module	Receives X, Y and Z acceleration values, calculates the magnitude and applies the sliding-window, threshold, spike and cooldown conditions.
4	Alert Generation Module	Plays the looping warning alarm and provides the confirmation period in which a false alert can be cancelled.
5	Location Tracking Module	Checks location services and permission, retrieves a high-accuracy position and builds the Google Maps link.
6	Emergency Message Module	Prepares the emergency message and opens the SMS composer; keeps the message as unsent data if the SMS action cannot be launched.
7	Local Storage and Permission Module	Stores user details, detection state and unsent message in SharedPreferences and manages location and messaging permissions.

Three of the modules are shown through the screens of the application (Figures 5.1 to 5.4). Each figure is followed by three paragraphs that explain the purpose of the screen, the elements that are visible in it, and the interaction and system response that it illustrates. Where a figure contains several panels, they are labelled (a), (b) and (c) from left to right.

5.1.1 User Details and Settings Module

This module collects the user name, blood group and emergency contact number. The source implementation loads previously stored values when the page opens and validates that all required fields are filled before saving. SharedPreferences is used to persist the values. The page can later be reopened to edit the information.

The source code shown in Figure 10.1 implements the module as a stateful page with three text controllers and a form key. When the page is created it loads the stored values for the keys `user_name`, `blood_group` and `emergency_number`, using an empty string when nothing has been stored. When the user presses Save, the form is validated, the three values are trimmed and written back to SharedPreferences, and the page closes.

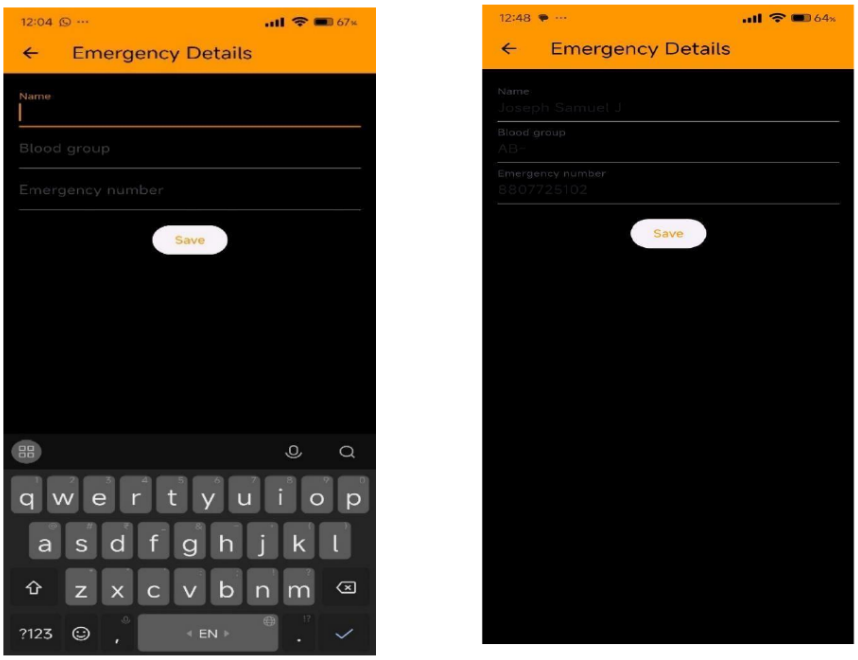


Figure 5.1: Registration / Emergency Details Screen — (a) empty form with keyboard, (b) stored details

Figure 5.1 shows the Emergency Details screen. It is the first screen that a new user must complete and it can be reopened later from the Edit Details button of the main screen. Its role in the application is to collect the three items that the emergency message needs, namely the name, the blood group and the emergency contact number, before detection can be enabled. Panel (a) shows the screen as it appears when it is opened with nothing stored, and panel (b) shows it after details have been stored.

Both panels show an orange title bar with a back arrow and the title Emergency Details. Below it are three text fields labelled Name, Blood group and Emergency number, and a rounded Save button. In panel (a) the Name field is active with a visible cursor, the other two fields show only their labels, and the Android on-screen keyboard occupies the lower part of the screen. In panel (b) the three fields contain a sample name, a blood-group value and a ten-digit number in a dim text colour on the black background. The status bars show the time and battery level of the test phone.

The user taps a field, types the value and presses Save. According to the source code, the form is validated, the values are written to SharedPreferences and the screen closes; if a required field is empty, the application shows the message "Please fill all fields" (Table 6.1, case 5). When the screen is opened again through Edit Details, the stored values are loaded back into the fields as in panel (b). The screen therefore guarantees that the alert workflow has a recipient and the personal information that a responder would need, and it keeps all of this data on the device.

5.1.2 Detection Control Module

The main screen provides an Accident Detection switch. If detection is enabled before the required user details are available, the app displays a warning and redirects the user to the details screen. The detection state is stored so that the application can restore it. When enabled, the app starts listening to accelerometer events.

The same screen displays the stored details so that the user can check them before a journey. The detection state is kept in SharedPreferences, and the button caption changes between Start Detection and Stop Detection to show the current state.

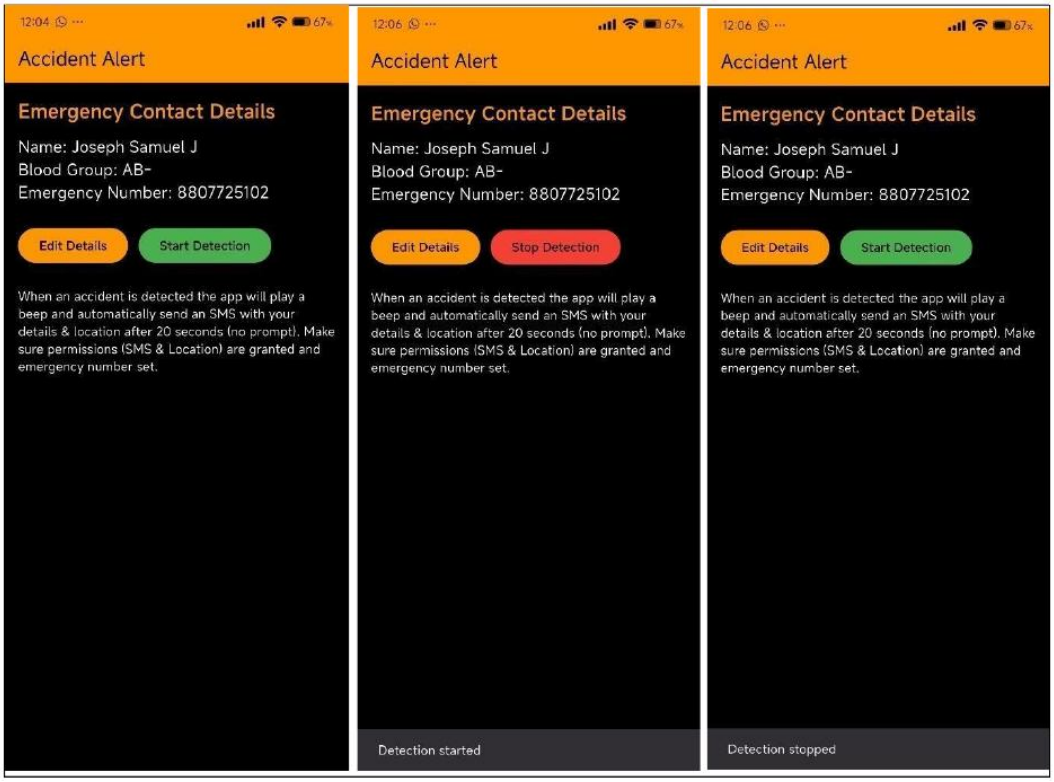


Figure 5.2: Main Accident Detection Screen — (a) detection off, (b) detection started, (c) detection stopped

Figure 5.2 shows the main screen, titled Accident Alert. It is the screen on which the application opens once the details have been saved, and it is the only screen that the user

needs while travelling. Its role is to display the stored emergency details, to switch sensor monitoring on and off, and to give a short statement of what the application will do after an accident is detected. Panel (a) shows detection switched off, panel (b) shows it after it has been started, and panel (c) shows it after it has been stopped again.

In every panel an orange title bar carries the heading Accident Alert, and an orange sub-heading reads Emergency Contact Details above three lines that give the name, the blood group and the emergency number. Two rounded buttons follow: Edit Details (orange) and a second button that is green and labelled Start Detection in panels (a) and (c) and red and labelled Stop Detection in panel (b). A text note states that when an accident is detected the app will play a beep and send an SMS with the user's details and location after 20 seconds (no prompt), and reminds the user to grant the SMS and Location permissions and to set the emergency number. A dark message bar at the bottom of panels (b) and (c) reads "Detection started" and "Detection stopped" respectively.

The user taps Start Detection; the application subscribes to the sensor streams, the button changes to the red Stop Detection state and the message "Detection started" confirms the change. Tapping Stop Detection ends monitoring and shows "Detection stopped". Edit Details opens the screen of Figure 5.1. The note on the screen describes the escalation that follows an accident; the countdown window in which the alert can be cancelled is not itself captured in the supplied screenshots, and a screenshot of it is suggested in Section 10.4. The wording "(no prompt)" and its relation to the SMS mechanism are discussed in Section 6.4.

5.1.3 Sensor Monitoring Module

The sensor module receives acceleration values along the X, Y and Z axes. The total magnitude is calculated as the square root of $x^2 + y^2 + z^2$. Recent magnitudes are stored in a small sliding window. The implementation compares the current magnitude with both an absolute threshold and the recent average, allowing it to react to a large absolute event or a sudden spike. A cooldown interval is used so that a single impact does not immediately trigger several repeated alerts. The original source uses a window size of eight readings, an absolute threshold of 25.0, a spike delta of 12.0 and an eight-second cooldown. These are prototype values and may require calibration for different devices.

The source snapshot (Figures 10.2 and 10.4) shows that the module subscribes to both the accelerometer and the gyroscope. For each event the magnitude of the three axis values is computed and passed to a motion-detector object together with the time of the reading. If

the gyroscope stream reports an error, the module marks the gyroscope as unavailable and continues with accelerometer-only detection. The detection values of the source snapshot are compared with the documented values in Section 5.2.

5.1.4 Alert Generation Module

When the detection condition is satisfied, the alert module plays a looping audio file to attract the user's attention. The documented workflow includes a confirmation period in which the user can dismiss a false event. If the user does not respond, the workflow proceeds to obtain location and prepare the emergency message. This confirmation stage is an important design control because smartphone motion alone cannot perfectly distinguish every crash from every non-crash movement.

In the source snapshot the alert state is held in a few flags (alert in progress, alert cancelled and sending alert), a countdown value that starts at 20 seconds and is shared with the countdown dialog through a `ValueNotifier`, an audio player for the warning sound, and two timers: one for the repeating beep and one for the countdown (Figure 10.3). While an alert is in progress, further motion events are ignored, which prevents a second alert from starting on top of the first.

When the user cancels, the module marks the alert as cancelled, stores the time so that the cooldown applies from that moment, cancels both timers, resets the motion detector, stops the sound and closes the countdown dialog (Figure 10.5). A local notification plug-in is also initialised in this part of the application (Figure 10.3).

5.1.5 Location Tracking Module

The location module first checks whether location services are enabled and then verifies the application permission. When required, it requests permission and retrieves a high-accuracy position through the Geolocator package. Latitude and longitude are converted into a Google Maps query URL. This representation is useful because the emergency contact can open the link directly on a mapping application.

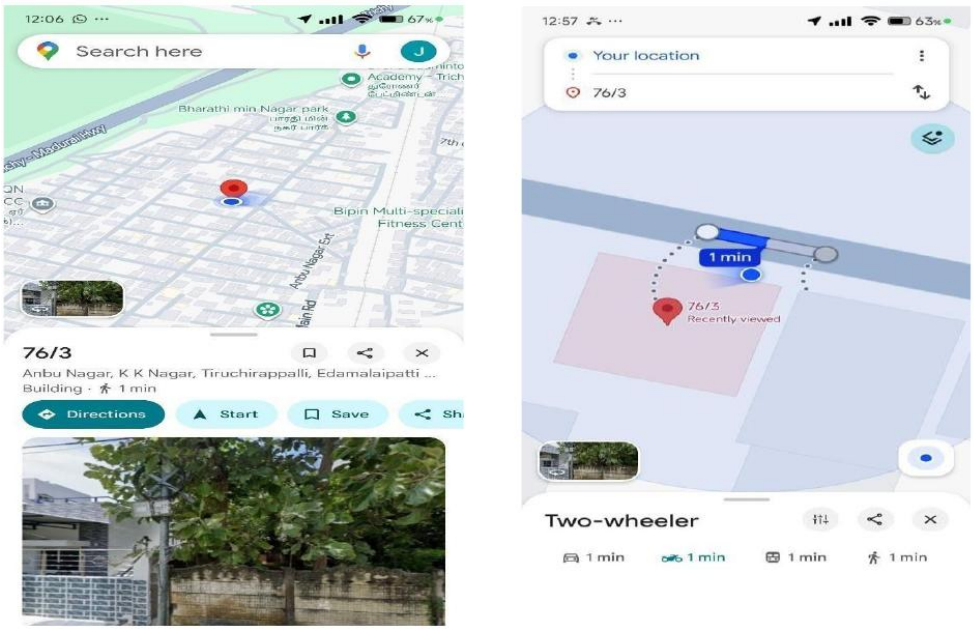


Figure 5.3: Live Location / Google Maps Output — (a) map view of the location link, (b) route view

Figure 5.3 shows the location output that results from the Google Maps link carried in the emergency message. It is the screen that the recipient of the message, or the tester, sees after tapping the link, and its role is to turn two coordinates into a place that can be recognised and navigated to. It is therefore reached at the end of the emergency workflow, after the message of Figure 5.4 has been received. Panel (a) shows the map view and panel (b) shows the route view.

Panel (a) shows a map with a search box labelled "Search here", a red pin at the selected location with a blue dot that marks the current position of the phone, labels of nearby places, and a place card at the bottom. The card gives a short label for the selected building, an address line, a walking-time indicator and the buttons Directions, Start, Save and Share, with a street-level photograph below. Panel (b) shows the route view with the fields "Your location" and a destination, a swap icon, a short route line marked "1 min", the label "Two-wheeler", and a row of travel modes (car, two-wheeler, public transport and walking) that each show "1 min". The destination pin carries the label "Recently viewed".

The recipient taps the link in the message, and the mapping application opens at the coordinates; pressing Directions or Start gives a route to the person. In the figure the red pin and the blue dot almost coincide and every travel estimate is one minute, which is consistent with a test carried out with the phone at the pinned place. This agrees with the location test of Table 6.3 (case 4), but it is a single observation and not a measurement of positional accuracy, which depends on GPS conditions. The figure contributes to

emergency handling because it shows that a link built only from latitude and longitude is sufficient for a recipient to find the sender.

5.1.6 Emergency Message Module

The emergency message contains an accident alert followed by available user details and the Google Maps location link. The source project uses an SMS URI so that the phone can open the SMS composer with the recipient and body prepared. If the SMS action cannot be launched, the message text is stored locally as an unsent message and can later be reopened.

The message has a fixed layout: a warning symbol and the words "Accident detected!", followed by the lines Name, Blood Group and Location, where the location line is a Google. Because the message is plain text, it can be read on any phone, and the link is converted into a tappable address by most messaging applications.

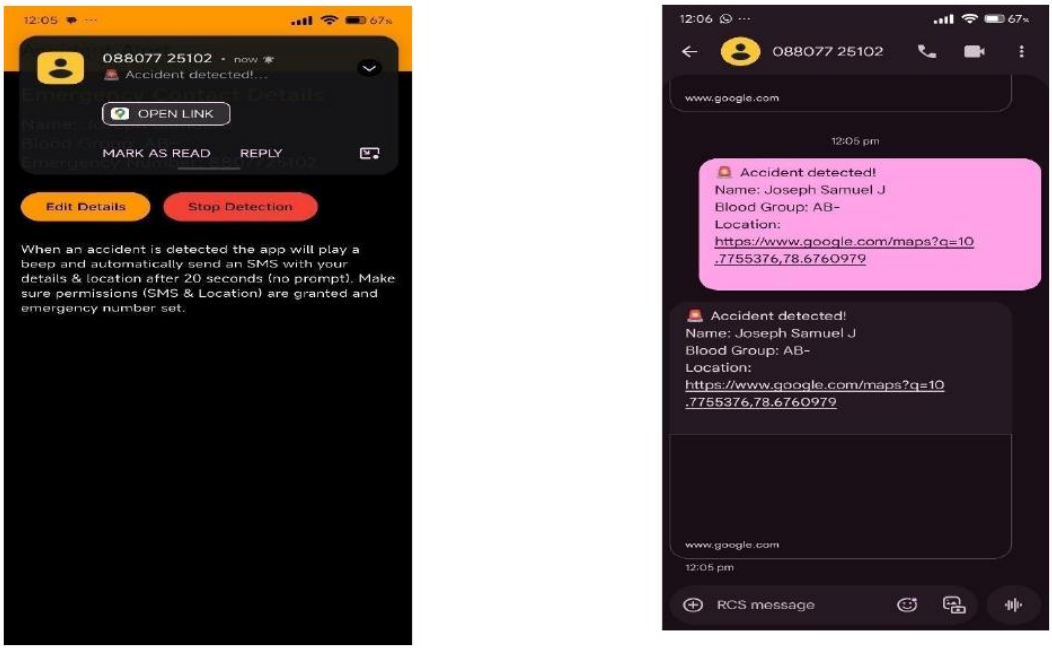


Figure 5.4: Emergency Message Notification / SMS Output — (a) notification, (b) message thread

Figure 5.4 shows how the emergency message appears on the phone. It is the output of the whole workflow and is reached after the countdown has ended without cancellation and the position has been obtained. Panel (a) shows the notification that announces the message while detection is still running, and panel (b) shows the message inside the messaging application.

In panel (a) a notification card shows a sender number, the time label "now", the beginning of the text "Accident detected!", an OPEN LINK button with a maps icon, and the actions MARK AS READ and REPLY. The main screen of the application is visible dimmed

behind it with the buttons Edit Details and Stop Detection and the explanatory note. In panel (b) the conversation screen has a back arrow, the number as the title, and call, video and menu icons. It contains the message text twice, once in a coloured bubble and once in a darker bubble with a link-preview footer, each with the siren symbol, "Accident detected!", Name, Blood Group and Location with the Google Maps address. Timestamps read 12:05 pm and the input box shows the hint "RCS message".

The recipient can tap OPEN LINK or the address to open the map of Figure 5.3. The screenshot confirms that the message content agrees with the design (accident notice, name, blood group and a Maps link), which supports test cases 1, 2 and 4 of Table 6.3. It does not show which mechanism launched the message, and it does not by itself prove delivery to a separate handset, because the number in the thread is the same as the saved emergency number. Section 6.4 discusses the difference between opening the SMS composer and sending automatically.

5.1.7 Local Storage and Permission Module

SharedPreferences stores the user name, blood group, emergency number, detection state and any unsent message. The permission handler and Android manifest manage access to location and messaging-related capabilities. Local storage is intentionally minimal and does not require an external database server.

Local storage keeps the application independent of a server, which also removes the need for user accounts or network authentication. The permission part of the module checks whether location services are enabled and whether the application has permission, and requests permission when it is denied, as shown in Figure 10.5. If permission is refused, the location step cannot be completed, which is one of the limitations discussed in Chapter 7.

5.2 Algorithms

The application uses a small set of deterministic algorithms that run entirely on the device. They cover sensor data collection, acceleration magnitude calculation, sliding-window spike detection, cooldown control, emergency escalation with false-alert cancellation, location retrieval and emergency message generation. The detection logic is threshold-based and does not use machine learning.

5.2.1 Sensor Data Collection and Acceleration Magnitude Calculation

When detection mode is enabled, the application subscribes to the accelerometer event stream provided by the sensors_plus package. Each event supplies acceleration values along the X, Y and Z axes of the phone.

For each accelerometer event, the magnitude is computed as $M = \sqrt{(x^2 + y^2 + z^2)}$. This converts three-axis motion into a single scalar value that can be compared with a threshold. Because gravity contributes to the magnitude, the chosen threshold must be significantly above ordinary stationary or low-motion readings.

$$M = \sqrt{(x^2 + y^2 + z^2)}$$

The gyroscope magnitude is calculated in the same way from the angular velocities about the three axes, and it is expressed in rad/s. A value of 1.0 rad/s corresponds to about 57° per second, so a gyroscope magnitude of 1.8 rad/s represents a rotation of about 103° per second.

5.2.2 Sliding-Window Spike Detection

The most recent values are maintained in a fixed-size window. The average of the window provides a short-term baseline. A possible accident is detected when either the absolute magnitude exceeds the absolute threshold or the difference between the current magnitude and the recent average exceeds a spike threshold. In conceptual pseudocode: read x, y, z; calculate magnitude; update window; calculate average; if magnitude > absoluteThreshold OR magnitude - average > spikeDelta, then evaluate cooldown and trigger the alert.

The steps of the documented detection algorithm are as follows.

1. Read the acceleration values x, y and z from the accelerometer event.
2. Calculate the magnitude $M = \sqrt{(x^2 + y^2 + z^2)}$.
3. Append M to the sliding window; if the window exceeds eight readings, remove the oldest reading.
4. Calculate the average of the values in the window as the short-term baseline.
5. Check the absolute condition: magnitude is greater than the absolute threshold.
6. Check the spike condition: magnitude minus the window average is greater than the spike delta.
7. If either condition is true, evaluate the cooldown and, if it has elapsed, trigger the alert.

Algorithm 5.1: Documented Sliding-Window Detection

```
ON each accelerometer event (x, y, z) at time t:
  M <- sqrt(x*x + y*y + z*z)
  append M to window; if size(window) > 8: remove oldest
  avg <- mean(window)
  IF M > 25.0 OR (M - avg) > 12.0 THEN
    IF (t - lastAlertTime) >= 8000 ms THEN
      lastAlertTime <- t
      startAlert()
```

Table 5.2: Prototype Detection Parameters

Parameter	Value	Purpose
windowSize	8 readings	Number of recent magnitudes kept to estimate the short-term average
absoluteThreshold	25.0	Magnitude above which an event is treated as a possible accident
spikeDelta	12.0	Difference between the current magnitude and the recent average that indicates a sudden spike
cooldownMs	8000 ms (8 seconds)	Minimum interval between two accepted detections

These are prototype calibration values and may require calibration for different devices.

5.2.3 Cooldown Logic

The detection timestamp is compared with the previous event timestamp. A new event is accepted only when the configured cooldown has elapsed. This prevents one physical impact from generating many rapid alerts from successive high sensor samples.

The cooldown is also applied after a cancelled alert. In the source snapshot the time of the last alert is set again when the user cancels (Figure 10.5), so a second alert cannot start immediately after the user has dismissed one. While an alert is in progress, events are ignored altogether.

5.2.4 Emergency Escalation Logic and False-Alert Cancellation

After a suspected accident, the alarm is started and the user is given a chance to cancel. If the alert is cancelled, no emergency message is sent and monitoring continues. If the warning expires, the application retrieves GPS coordinates, constructs the message and initiates the SMS action. The documented mini-project requirement specifies a 20-second confirmation window. The source snapshot of Figure 10.3 initialises the countdown to 20 seconds, which agrees with this requirement; the remaining timing question is discussed in Section 6.4.

1. Start the looping warning alarm and display the warning to the user.
2. Give the user a confirmation window in which the alert can be cancelled.

3. If the alert is cancelled, stop the alarm; no emergency message is sent and monitoring continues.
4. If the warning expires without cancellation, proceed to location retrieval and emergency message generation.

5.2.5 Location Retrieval and Emergency Message Generation

After escalation, the application obtains the current position and prepares the emergency message that is addressed to the saved emergency contact.

1. Check whether location services are enabled and verify the application permission; request permission when required.
2. Retrieve a high-accuracy position through the Geolocator package.
3. Convert the latitude and longitude into a Google Maps query URL.
4. Compose the emergency message containing the accident notice, the stored name and blood group, and the Google Maps link.
5. Encode the message in an SMS URI addressed to the stored emergency number and open the SMS action.
6. If the SMS action cannot be launched, store the message locally as an unsent message so that it can be reopened later.

Algorithm 5.2: Escalation, Cancellation and Message Preparation

```
startAlert():
  alertInProgress <- true; alertCancelled <- false
  start looping warning sound; show countdown (20 s)
  IF user presses Cancel THEN
    alertCancelled <- true; stop sound and timers
    reset detector; return to monitoring
  WHEN countdown reaches 0 AND NOT alertCancelled: sendEmergencyAlert()

sendEmergencyAlert():
  IF location service enabled: check permission, request if denied
  position <- current high-accuracy position (if available)
  link <- "https://www.google.com/maps?q=" + lat + "," + lon
  message <- accident notice + name + blood group + link
  address message to saved emergency number; launch SMS action
  IF launch fails: store message as unsent
```

5.2.6 Detector Visible in the Source Snapshot

The source screenshots of Chapter 10 show a motion-detector class that differs in its numbers from the documented parameters of Table 5.2. Its parameters are listed in Table 5.3. An accelerometer sample is stored as an impact when the magnitude is at least 28.0, or at least 20.0 with a change of at least 6.0 from the previous reading, or at least 18.0 with a change of at least 14.0. A gyroscope sample is stored as a rotation when its magnitude is at

least 1.8. Samples are kept with their time stamps in two lists, and after every new sample a private method named `_checkForAccident` is called with the time of the sample.

Table 5.3: Detection Parameters Visible in the Source Snapshot

Parameter	Value	Meaning
_impactMagnitude	20.0 m/s² (about 2.0 g)	Strong impact when combined with a change of 6.0
_extremeMagnitude	28.0 m/s² (about 2.9 g)	Impact on its own
Change test	6.0 (with 20.0) and 14.0 (with 18.0)	Absolute difference from the previous magnitude
_rotationMagnitude	1.8 rad/s (about 103°/s)	Significant rotation reported by the gyroscope
_impactWindow	650 ms	Time window for impact samples
_rotationWindow	1200 ms	Time window for rotation samples
_sensorCooldownMs	8000 ms	Cooldown between alerts (same as documented)
_countdownSeconds	20 s	Warning countdown (same as documented)

The body of `_checkForAccident` is not visible in the supplied screenshots. For that reason this report does not state how the impact and rotation lists are combined, for example whether both must be present inside their windows, and it makes no claim about the detection accuracy of the snapshot.

Both sets of values express the same two ideas, an absolute magnitude test and a sudden-change test, but with different numbers, and the snapshot adds a rotation test and time windows in place of the eight-reading average. The report preserves both: Table 5.2 is the documented prototype specification and Table 5.3 is what the source screenshots show. The cooldown of 8 seconds and the 20-second countdown are the same in both. The author should confirm which build is demonstrated at the examination and treat the matching table as authoritative.

5.3 Architecture

The architecture is entirely mobile-centric. The Flutter UI communicates with an accident-detection controller. Sensor events enter from sensors_plus; configuration and user data are read from SharedPreferences; the controller activates audio, requests the GPS location, and invokes the SMS/URL mechanism. No external application server is required for the core prototype.

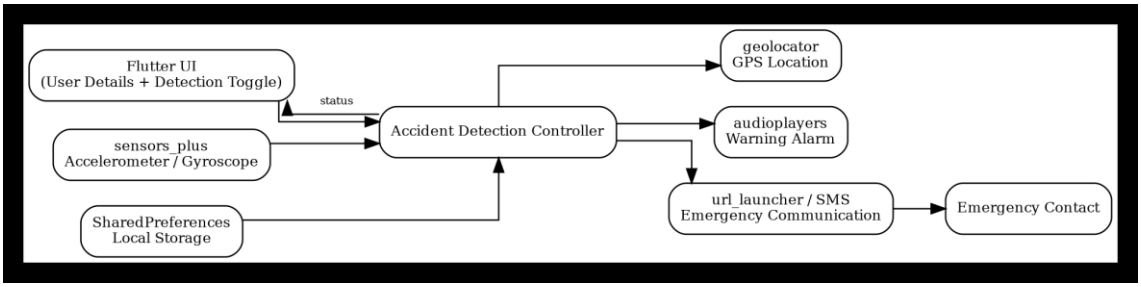


Figure 5.5: System Architecture of Accident Detection and Alert System

The flow of control through the architecture is: the User enables detection on the Flutter UI; the Sensor Monitoring component reads accelerometer events; the Detection Algorithm evaluates the threshold, spike and cooldown conditions; the Warning/Confirmation stage plays the alarm and allows cancellation; the GPS Location component retrieves the coordinates; and the Emergency Message/SMS component addresses the message to the Emergency Contact. SharedPreferences provides local storage for user details, detection state and any unsent message throughout this flow.

The architectural components are as follows.

- **User and Flutter UI:** the user enters details and controls the detection switch through the Flutter interface.
- **Accident-detection controller:** coordinates sensor events, audio, location and the SMS/URL mechanism.
- **Sensor input:** accelerometer events are supplied through sensors_plus.
- **Local storage:** configuration and user data are read from and written to SharedPreferences.
- **Output:** the controller activates the warning audio, requests the GPS location and invokes the SMS/URL mechanism to reach the emergency contact.

No external application server is required for the core prototype.

5.4 Case Study

Consider a user named Sam who has saved blood group O+ and an emergency contact number. Before beginning a journey, Sam enables Accident Detection. The application subscribes to the accelerometer stream and continuously calculates motion magnitude. Normal vehicle vibration remains below the configured conditions, so no warning is shown.

A sudden high-impact movement causes the magnitude to exceed the detection condition. The application starts the alarm and displays the warning. If Sam is safe and the event was

caused by a pothole or accidental phone drop, the alert can be cancelled. No message is generated, and the system returns to monitoring.

If Sam is unable to respond, the warning expires. The app checks location permission, obtains the current position, and creates an emergency message such as "Accident detected! Name: Sam; Blood Group: O+; Location: [Google Maps link]." The recipient can open the link to view the location. This scenario illustrates the intended value of the application: reducing dependence on the victim's ability to manually describe the situation and location.

The case study also illustrates the key limitation of threshold detection. A sufficiently violent non-accident motion may trigger the same sequence. The cancellation stage therefore acts as a practical safeguard, while future machine-learning and context-aware methods can improve automatic discrimination.

CHAPTER 6
IMPLEMENTATION DETAILS

6.1 Implementation Procedures

The project is implemented as a Flutter application. Development begins with creation of the Flutter project and configuration of dependencies in pubspec.yaml. The sensor, geolocation, local-storage, audio and permission packages are then installed using flutter pub get. Required Android permissions are declared in AndroidManifest.xml.

The user-details screen is implemented first so that personal data and the emergency number can be entered, validated and stored. The application startup code checks SharedPreferences to decide whether the details page or main detection page should be displayed. This prevents accident monitoring from being activated before an emergency recipient is configured.

The main screen implements the detection toggle. When enabled, a sensor listener is started. Each accelerometer event is converted into a magnitude, appended to a small window and compared with the absolute and relative spike conditions. When a qualifying event occurs and the cooldown has elapsed, the alert routine is called.

The alert routine starts the audio warning, loads stored user details and checks location availability and permission. A current GPS position is requested and inserted into a Google Maps link. The message is encoded into an SMS URI for the stored emergency number. If the phone cannot open the SMS action, the message is retained locally as an unsent message and a button is provided to retry later.

Implementation is validated on a physical smartphone because accelerometer, gyroscope, GPS and SMS behaviour cannot be fully reproduced by a desktop environment. Threshold testing is performed carefully using controlled movements rather than dangerous real-world crash simulation.

The implementation environment consists of Flutter and Dart for the application, sensors_plus for accelerometer events, geolocator for location, SharedPreferences for local storage, audioplayers for the warning sound, permission_handler for runtime permissions, url_launcher for the SMS/URL mechanism, and the Android SDK with Gradle as the build tools. The user interface of the principal screens is shown in Figures 5.1 to 5.4 of Chapter 5: the Registration / Emergency Details screen (Figure 5.1), the Main Accident Detection

screen (Figure 5.2), the Live Location output (Figure 5.3) and the Emergency Message notification (Figure 5.4).

6.1.1 Project Configuration and Package Integration

The packages of Table 2.4 are declared in pubspec.yaml and downloaded with flutter pub get. The explorer panel of the editor in Figures 10.1 to 10.5 lists the Android plug-in folders that were generated in the build directory of the project (named detect): flutter_local_notifications,geolocator_android,package_info_plus,path_provider_android, permission_handler_android,sensors_plus,shared_preferences_androidurl_launcher.The application code is in two files, lib/main.dart and lib/user_details_page.dart. Each plug-in is used only in the module that needs it, so the sensor package is used by the detection code, the location package by the escalation code and the storage package by both pages.

6.1.2 Permissions, Storage and Device Services

Location and messaging permissions are declared for the Android platform and requested at run time. The escalation code first calls Geolocator.isLocationServiceEnabled(), then Geolocator.checkPermission(), and calls Geolocator.requestPermission() when the permission has been denied (Figure 10.5). The user details are read and written through SharedPreferences.getInstance() (Figure 10.1). The warning sound is controlled through an audio-player object, and the cancellation code stops it inside a try block so that a failure to stop the sound is logged and does not stop the rest of the cancellation.

6.1.3 Order of Implementation

The modules were implemented in the order that their dependencies require: user details and storage first, then the main screen and the detection switch, then sensor processing and the detection rule, then the alarm and cancellation, and finally location retrieval and message preparation. Each stage could be run on the phone before the next one was added, which is the staged development described in Section 2.6.4.

6.2 Testing Methods and Results

Testing was carried out to verify that the modules of the application function correctly and work together as expected. Because the application depends on real sensor, GPS and messaging behaviour, the testing strategies described below were applied to the prototype, and the resulting test cases are tabulated in Section 6.3.

- **Functional Testing:** Verifies saving user details, detection control, sensor-event handling, warning generation, cancellation behaviour, location retrieval and emergency-message creation.

- **Usability Testing:** Checks whether labels, input fields, buttons and warnings are understandable and whether the workflow can be completed without technical knowledge.
- **Device and Compatibility Testing:** Runs the application on Android devices with different screen sizes and sensor characteristics. Permission behaviour is verified for the target Android version.
- **Location Testing:** Compares the generated Google Maps link with the physical test location and verifies permission-denied and location-disabled conditions.
- **Emergency Message Testing:** Checks recipient number, message content, encoded location link and the unsent-message fallback when an SMS action cannot be launched.
- **Persistence Testing:** Closes and reopens the app to confirm that personal details and detection state are stored as expected.
- **Safety-Oriented Testing:** Avoids real collisions or hazardous driving. Sensor events are simulated by controlled hand movements or low-risk bench tests.

The test cases for the user details page, accident detection and SMS/location workflow were executed on the prototype, and the status recorded for each case in Tables 6.1 to 6.3 is Pass.

6.2.1 Test Environment and Data

The tests were run on a physical Android smartphone, because an emulator cannot reproduce real sensor behaviour (Section 2.3). Abnormal motion was produced by controlled hand movements and not by real collisions. Each test case compares an expected result with the observed result and records the status. The sample details in the tables (for example the name Sam and the number 9876543210) are illustrative test data; the screenshots of Chapter 5 were captured in a separate run with other sample values.

6.3 Test Cases

6.3.1 User Details Page Test Cases

Table 6.1: User Details Page Test Cases

ID	Scenario	Steps / Data	Expected Result	Actual Result	Status
1	Enter user name	Type Name = Sam	Name is saved and visible	Name saved and visible	Pass
2	Enter blood group	Enter/select O+	Blood group is saved	Saved successfully	Pass
3	Enter emergency number	Enter 9876543210	Number is saved and retrievable	Saved and retrievable	Pass

ID	Scenario	Steps / Data	Expected Result	Actual Result	Status
4	Save details	Provide all fields and tap Save	Data persists and is usable by alert workflow	Data saved and persists	Pass
5	Empty fields	Tap Save with required field empty	App shows “Please fill all fields”	Validation message shown	Pass

6.3.2 Accident Detection Test Cases

Table 6.2: Accident Detection Test Cases

ID	Scenario	Action / Data	Expected Result	Actual Result	Status
1	Enable detection	Turn ON Accident Detect	Sensor monitoring starts	Monitoring started	Pass
2	Disable detection	Turn OFF detection	Monitoring stops / no new alerts	Monitoring stopped	Pass
3	Normal movement	Walk / normal drive-like movement	No accident alert	No alert triggered	Pass
4	Minor shake	Controlled small jerk	Warning may appear according to threshold; can be dismissed	Popup / warning displayed	Pass
5	Dismiss false alert	Cancel warning	Alarm stops and no SMS is sent	Alarm stopped; no SMS	Pass
6	Major shake	Controlled stronger simulated movement	Alarm and warning countdown start	Alarm started and warning shown	Pass
7	Ignore alert	Do not dismiss	Emergency message generated after confirmation period	Message generated	Pass
8	Multiple shakes	Repeat rapid movement	Cooldown prevents repeated alerts for same event	Single event handled	Pass
9	Alarm sound	Allow warning to run	Warning audio continues until stopped by workflow	Audio played	Pass

6.3.3 SMS and Location Test Cases

Table 6.3: SMS and Location Test Cases

ID	Scenario	Expected Result	Actual Result	Status
1	Emergency message content	Contains accident notice, name, blood group and Google Maps location	Correct content generated	Pass
2	Message after timeout	Message workflow begins after user does not cancel warning	Workflow initiated	Pass
3	No message when cancelled	Cancellation prevents emergency escalation	No emergency message	Pass
4	Location accuracy	Latitude and longitude open correct Maps location	Location link correct	Pass
5	Location permission denied	App requests permission or prevents location step gracefully	Permission flow displayed	Pass
6	SMS action unavailable	Message is retained as unsent data for retry	Unsent message saved	Pass

All twenty executed test cases in Tables 6.1 to 6.3 are recorded with the status Pass. The tests check functional behaviour on the prototype; they are not a statistical evaluation, and the report does not derive an accuracy figure from them.

6.3.4 Proposed Additional Tests

The tests in Table 6.4 would strengthen the evaluation, but they have **not been executed** and no result is claimed for them. They are listed so that the verified results above are not confused with planned work.

Table 6.4: Proposed Additional Tests (Not Executed)

ID	Proposed test	Purpose	Status
P1	Run the application on several handset models	Observe sensor variation between devices	Proposed
P2	Repeat motion trials with the phone in a pocket, in a mount and loose on a seat	Study the effect of phone placement	Proposed
P3	Repeat controlled normal-motion trials over a longer period	Estimate how often false alerts occur	Proposed
P4	Measure battery use with detection on for a fixed period	Quantify battery consumption	Proposed
P5	Test with the screen locked or the application in the background	Check behaviour under Android background restrictions	Proposed
P6	Send the emergency message to a separate handset	Confirm delivery to a different recipient	Proposed
P7	Request a position in weak-signal places	Observe GPS delay and accuracy	Proposed
P8	Interrupt the countdown with a call or by switching applications	Check robustness of the countdown	Proposed

6.4 Implementation Consistency and SMS Mechanism Note

The project report describes a 20-second user confirmation period before emergency escalation. The source snapshot of Figure 10.3 initialises the countdown to 20 seconds, and the note on the main screen of Figure 5.2 also states 20 seconds, so these agree with the documented workflow. An earlier code excerpt used for this report contained a timer that stopped the alarm after 10 seconds and launched the SMS URI within the alert routine. The final submission build should be checked so that every timer, the screen text, the test cases and this report use the same 20-second value.

6.4.1 Opening the SMS Composer versus Sending Automatically

The report describes the message as encoded in an SMS URI and launched through url_launcher. A URI of this kind normally opens the phone's messaging application with the recipient and the text already filled in, and the user then presses send; sending a message with no user action needs an SMS-sending API and the corresponding permission, and is

subject to platform policy. The main screen, however, says that the message is sent "automatically ... (no prompt)", and Figure 5.4 shows the message in a message thread. The visible evidence therefore shows that the message reached the messaging application, but the screenshots do not show which mechanism did so.

For this reason the report uses the neutral wording "prepares and launches the SMS action", does not claim guaranteed automatic delivery, and does not claim emergency-service integration. The author should state the mechanism of the final build precisely in the presentation. Table 6.5 lists the points that should be confirmed.

Table 6.5: Documented Values Compared with the Source Snapshot

Item	Documented in the report	Source snapshot / screenshots	Status
Confirmation window	20 seconds	_countdownSeconds = 20; screen text states 20 seconds	Consistent
Cooldown	8 seconds	_sensorCooldownMs = 8000	Consistent
Sensors	Accelerometer and gyroscope	Both streams are used; accelerometer-only fallback	Consistent
Detection thresholds	Window 8, absolute 25.0, spike 12.0	20.0 / 28.0 with changes of 6.0 and 14.0; rotation 1.8; windows 650 ms and 1200 ms	Differ; confirm
SMS mechanism	SMS URI; unsent-message fallback	Screen text says automatic; mechanism not visible	Confirm
Local notifications	Not listed originally	FlutterLocalNotificationsPlugin initialised	Now documented

CHAPTER 7
ADVANTAGES AND DISADVANTAGES

7.1 Advantages

The Accident Detection and Alert System brings detection, location and emergency communication into a single smartphone application. Its main advantages are:

- **Low hardware cost.** The solution uses sensors that are already present in a smartphone, so no dedicated detector, microcontroller or GPS receiver has to be bought or installed in the vehicle.
- **Reduced reporting delay.** Rapid detection and location-aware emergency communication can shorten the time between an accident and the first notification, especially when the person cannot call for help.
- **Simple interface.** Three fields and one switch are all that the user has to operate, which suits a safety tool that is configured once and then left running.
- **Local storage.** Essential settings are kept with SharedPreferences, so no external database or account is needed and no personal data has to be uploaded.
- **Easy-to-open location.** GPS coordinates are converted into a Google Maps link, which a recipient can open directly in a mapping application without copying numbers.
- **False alerts can be cancelled.** The 20-second countdown lets the user stop an alert caused by a pothole, hard braking or a dropped phone before any message is prepared.
- **Modular design.** Sensing, alerting, location, messaging and storage are separate modules, so the thresholds or a package can be changed without rewriting the whole application.
- **Wide range of users.** The approach can be useful to drivers, riders, cyclists and users in isolated environments, because it travels with the person.

7.2 Disadvantages

Although the system provides a low-cost safety mechanism, it has the following limitations:

- **False positives.** Threshold-based detection can produce false positives from phone drops, potholes or abrupt movements that are not accidents, and the user must then cancel the alert.

- **Sensor and placement variation.** Accuracy depends on phone placement, device sensor quality and calibration; the same event can give different readings in a pocket, on a mount or on a seat.
- **Cellular and permission dependence.** A message still depends on cellular service and on the permissions granted to the application; without them the message cannot be completed.
- **GPS limits.** GPS may be slow or inaccurate indoors, in tunnels or in areas with limited satellite visibility, so the position in the message may be missing or approximate.
- **Battery and background limits.** Continuous sensor monitoring can increase battery consumption, and modern Android background-execution policies may restrict monitoring when the application is not in the foreground. The prototype has not been measured for either effect.
- **No vehicle data.** A smartphone-only solution cannot access vehicle crash data unless additional integration is added.
- **No emergency-service link.** The prototype does not contact the nearest hospital or a government emergency dispatch system; it only prepares a message for a saved contact.

CHAPTER 8
RESULT AND DISCUSSION

8.1 Functional Results

The implemented system successfully demonstrates the complete prototype workflow: capturing and saving emergency details, enabling sensor monitoring, detecting high-motion events, producing a warning, obtaining location and generating an emergency message with a Google Maps link. The user-detail validation and local-storage behaviour support repeated use without re-entering information on every launch.

Table 8.1: Functional Result Summary

Function	Observed Result	Status
User details storage	Name, blood group and emergency number stored locally	Pass
Detection toggle	Monitoring can be enabled/disabled	Pass
Normal movement handling	No alert during ordinary controlled movement	Pass
Abnormal motion handling	Warning generated under strong simulated motion	Pass
False-alert handling	User can cancel before emergency action	Pass
GPS location	Coordinates embedded in Maps link	Pass
Emergency message	Personal data and location included	Pass
Persistence	Saved details available after reopening	Pass

These results are taken from the test cases of Chapter 6. They describe the observed behaviour of the prototype under controlled movements and are not accuracy measurements.

8.1.1 Evidence Provided by the Screenshots

The screenshots of Chapter 5 and the source screenshots of Chapter 10 are the visual evidence for the prototype. The figures show the entry of details, the detection switch, the map link, the notification and message, and the source code; they do not show a measured detection accuracy or the countdown dialog.

8.2 Discussion of Detection Behaviour

The use of both an absolute magnitude threshold and a difference from the recent average provides a simple form of contextual detection. The absolute threshold reacts to clearly large acceleration, while the spike condition reacts to a sudden deviation from recent motion. The sliding window smooths the baseline and the cooldown prevents repeated

triggers from the same event. These choices make the algorithm easy to explain and suitable for an academic prototype.

However, a deterministic threshold does not provide a universal definition of an accident. Sensor output differs across devices, and the same physical event can produce different readings depending on whether the phone is fixed to the vehicle, carried in a pocket or placed loosely on a seat. The confirmation alarm is therefore an essential part of the prototype rather than merely a user-interface feature.

The source snapshot refines the idea in three ways: it keeps impact and rotation samples inside short time windows, it adds the gyroscope as a second source, and it falls back to accelerometer-only detection when the gyroscope is unavailable. The extra rotation condition can help to separate a straight, hard stop from a tumbling event, but the report does not claim that this improves accuracy, because no comparative measurement has been made.

8.3 Discussion of Emergency Communication

The location-aware message is the most practically useful output of the project. Latitude and longitude alone may be difficult for a recipient to interpret quickly, while a Google Maps link can be opened directly. Including the user's name and blood group adds context. The unsent-message fallback is also valuable because it avoids losing the emergency text when the SMS action cannot be opened immediately.

The current approach should be understood as a prototype emergency aid rather than a certified emergency dispatch mechanism. Direct SMS transmission may be restricted by platform policy or require user interaction depending on Android version and package behaviour. A production system would require a carefully tested communication strategy, clear consent, privacy safeguards and integration with local emergency-service rules.

Figure 5.4 shows that the messaging application on the test phone offered RCS messaging, which is a reminder that the way in which a text is carried depends on the phone and the network. For an emergency alert, the plain SMS path is the more dependable choice because it needs only cellular coverage, and a production version would have to make this choice explicit.

8.4 Overall Analysis

The project meets its primary educational and functional objective by integrating several mobile-device capabilities into one safety workflow. It shows that a smartphone can serve

as a compact sensing, processing, location and communication platform. The solution is especially attractive for a mini project because the architecture is visible and explainable: data enters from sensors, a detection condition is evaluated, the user is warned, location is obtained, and a message is generated.

The literature review indicates that advanced systems reduce false positives through sensor fusion, vehicle data, acoustic context or machine learning. The present implementation is therefore best viewed as a baseline system. Its value lies in simplicity, low deployment cost and a clear path for improvement. The most important next technical step would be to collect labelled normal-motion and simulated-impact data from multiple devices and replace fixed thresholds with a calibrated or learned classifier.

The observations of this chapter can be separated into demonstrated prototype behaviour, limitations and future improvements:

- **Demonstrated prototype behaviour:** user details are saved and validated; detection can be enabled and disabled; normal movement raises no alert; strong simulated motion produces a warning; a false alert can be cancelled before emergency action; the location is embedded in a Google Maps link; and the emergency message contains personal data and location.
- **Limitations:** the threshold-based detector is device-dependent and does not provide a certified accident-detection accuracy; emergency communication depends on SMS capability and platform restrictions; and the confirmation timing of the implementation snapshot must be aligned with the documented workflow.
- **Future improvements:** collection of labelled normal-motion and simulated-impact data from multiple devices to replace fixed thresholds with a calibrated or learned classifier, together with the enhancements listed in Section 9.3.

8.5 Limitations of the Evaluation

- **No quantitative accuracy.** The report gives no detection accuracy, false-positive rate or response time, because none was measured.
- **Controlled movements.** Accident-like motion was simulated by hand movements; no real or vehicle crash was used, for safety reasons.
- **Delivery not established.** The screenshots show the message in a thread, but delivery to a different handset and the time taken have not been tested.

CHAPTER 9
CONCLUSION

9.1 Summary

The Accident Detection and Alert System Using Smartphone Sensors was designed to reduce the delay between an accident and the first emergency notification. The application uses the smartphone accelerometer and gyroscope/motion sensing capability to identify sudden abnormal movement. It stores essential user data locally, provides a warning stage to handle false detections, obtains GPS coordinates and prepares an emergency message containing a Google Maps location link. The system continuously monitors sensor values and compares the detected motion patterns with predefined threshold conditions to identify possible accident situations. When an abnormal event is detected, the warning mechanism gives the user an opportunity to cancel the alert if the detection is caused by normal activities or accidental movement. This approach helps reduce unnecessary emergency notifications while maintaining a simple and responsive user interaction. The integration of sensor monitoring, location services and automated message preparation also demonstrates how multiple smartphone capabilities can be combined to create a practical safety-oriented application. The system is implemented with Flutter and Dart using mobile packages for sensing, location, local storage, audio and device actions.

The project demonstrates that useful emergency functionality can be created without specialized external hardware. It also illustrates important software-engineering concerns such as permission handling, state persistence, real-time event streams, threshold calibration and safe testing. Functional test cases confirm the intended prototype workflow for user details, detection control, alerts, location and message generation.

Each objective of Section 1.3 has been addressed by the prototype: the details screen, the detection switch, sensor monitoring, the warning and cancellation, GPS retrieval, the Maps link and local storage are all visible in the figures of Chapter 5 and Chapter 10.

9.2 Limitations

Threshold values are device-dependent and require calibration. Different smartphones may produce different sensor readings due to variations in hardware and sensor sensitivity. Therefore, suitable threshold values must be calibrated and tested for reliable accident detection.

Phone placement can influence measured acceleration and rotation. The position and orientation of the smartphone, such as in a pocket, bag, or vehicle, can change sensor readings and may affect detection accuracy.

The application cannot guarantee detection of every accident or rejection of every non-accident movement. The system relies on predefined sensor thresholds, so sudden braking, dropping the phone, or other movements may sometimes cause false detection, while some accidents may not produce sufficient sensor values.

Location availability depends on GPS conditions and user permission. GPS accuracy can be affected by signal strength and environmental conditions. If location permission is denied or GPS is unavailable, an accurate accident location may not be obtained.

Emergency communication depends on SMS/cellular capability and platform restrictions. Successful message delivery requires network connectivity, SMS availability, and appropriate device permissions. Poor network conditions may delay or prevent communication.

The current design stores only minimal data and does not maintain a full accident-history database. The prototype focuses on the core detection and alert workflow and does not currently provide long-term accident records, cloud synchronization, or detailed event history.

No direct hospital, ambulance, or government emergency API integration is included. The system prepares emergency information and a location link but does not directly connect with emergency-response organizations or external emergency-service APIs.

The documented confirmation duration and source-code timing values must be synchronized in the final build. The confirmation time described in the documentation should match the actual value implemented in the source code, test cases, and final application to avoid inconsistencies during evaluation.

9.3 Future Scope

Machine-learning classification using labelled accelerometer and gyroscope sequences to reduce false positives. A machine-learning model can be trained using real and simulated accident sensor data to distinguish genuine accident patterns from normal activities such as walking, running, sudden braking, or dropping the phone. This can improve the accuracy and reliability of the detection system.

Adaptive sensitivity that learns each user's usual movement patterns and phone placement. The system can learn the user's normal movement characteristics and typical smartphone placement over time. Based on this information, detection thresholds can be automatically adjusted to provide more personalized and accurate accident detection.

Multiple emergency contacts with prioritized escalation. The application can support several emergency contacts and notify them according to a predefined priority order. If the first contact does not respond within a specified period, the system could automatically proceed to the next contact.

Direct integration with local emergency numbers or approved emergency-service APIs where legally and technically supported. Future versions could connect directly with authorized ambulance, police, or emergency-response services. Such integration could allow accident information and location details to be transferred more quickly, subject to technical availability, legal requirements, and user consent.

Wearable-device integration for heart rate, oxygen saturation and fall-detection information. Smartwatches and other wearable devices could provide additional health and motion information. Combining these measurements with smartphone sensor data could provide a more comprehensive assessment of an accident situation.

Automatic voice prompt to determine whether the user is conscious and able to respond. After detecting a possible accident, the system could provide a voice prompt asking the user to confirm their condition. If the user does not respond within a specified time, the application could proceed with the emergency notification process.

Optional secure cloud history for accident analytics, subject to consent and privacy controls. Users could optionally store accident events securely in the cloud for future reference and analysis. Appropriate authentication, encryption, access control, and user consent would be required to protect sensitive information.

Detection of accident-prone locations and context-aware safety warnings. Historical accident information and location data could be analyzed to identify areas where accidents occur more frequently. The application could then provide appropriate safety warnings when the user enters such locations.

Battery-aware background monitoring and optimization for modern Android restrictions. Future development can improve background sensor monitoring while reducing battery consumption. The application could use efficient sensor sampling and

Android-compatible background services to maintain monitoring without significantly affecting device performance.

Vehicle integration through Bluetooth/OBD-II to incorporate speed, airbag or diagnostic information. Integration with vehicle systems could provide additional information such as vehicle speed, engine status, diagnostic data, or airbag activation. Combining this information with smartphone sensor readings could improve accident detection and provide more detailed emergency information.

9.4 Applications

- **Personal safety for drivers and passengers.** The system can provide an additional layer of safety by detecting unusual motion associated with possible accidents and preparing an emergency notification with the user's location. It can be useful for both drivers and passengers during regular travel.
- **Two-wheeler and bicycle safety where built-in automotive crash notification is unavailable.** Motorcycles and bicycles generally have fewer built-in safety and crash-notification features than modern cars. A smartphone-based system can provide a low-cost alternative for detecting possible accidents and initiating an alert.
- **Emergency support for users travelling through remote areas.** The application can be particularly useful when users travel through locations where immediate assistance may not be easily available. Sharing the user's GPS location with emergency contacts can help them identify the approximate accident location more quickly.
- **Fleet and field-worker safety when combined with organization-specific monitoring.** The system can be extended for delivery personnel, service technicians, transport workers, and other field employees. With suitable organizational monitoring and privacy controls, accident alerts could help supervisors respond to incidents involving employees.
- **Research platform for mobile sensor fusion and intelligent transportation studies.** The application can serve as a foundation for research involving accelerometer, gyroscope, GPS, and other smartphone sensor data. Researchers can use the platform to study movement classification, accident detection, sensor fusion, and intelligent transportation applications.
- **Foundation for smart-city emergency-response integration.** The system can potentially be integrated with larger smart-city platforms that coordinate location

information and emergency notifications. Such integration could support faster information sharing between users and authorized emergency-response systems.

- **Educational demonstration of Flutter, sensor streams, GPS, local storage and mobile communication services.** The project provides a practical example of how different mobile technologies can be combined to develop a real-world application. It can be used for demonstrating Flutter and Dart development, real-time sensor streams, GPS services, local data storage, permissions, alerts, and mobile communication features.

CHAPTER 10
APPENDICES

10.1 Sample Code

10.1.1 User Details Page

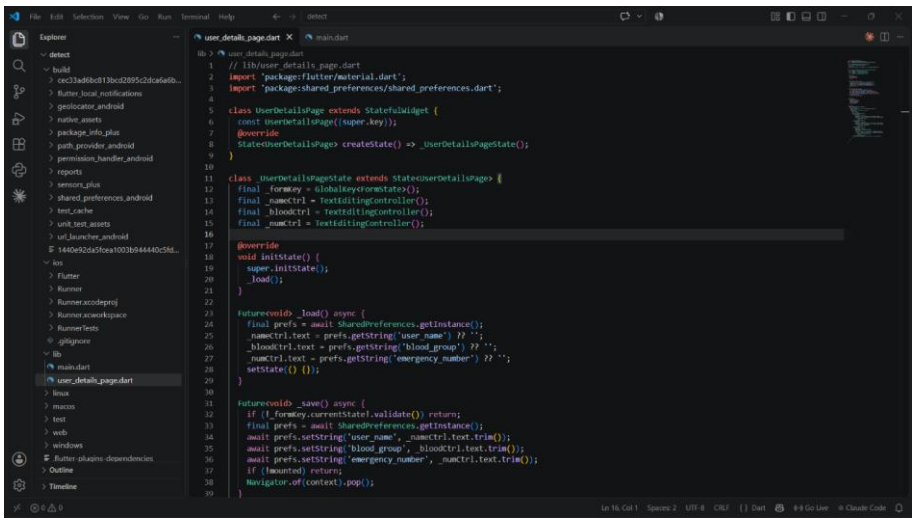


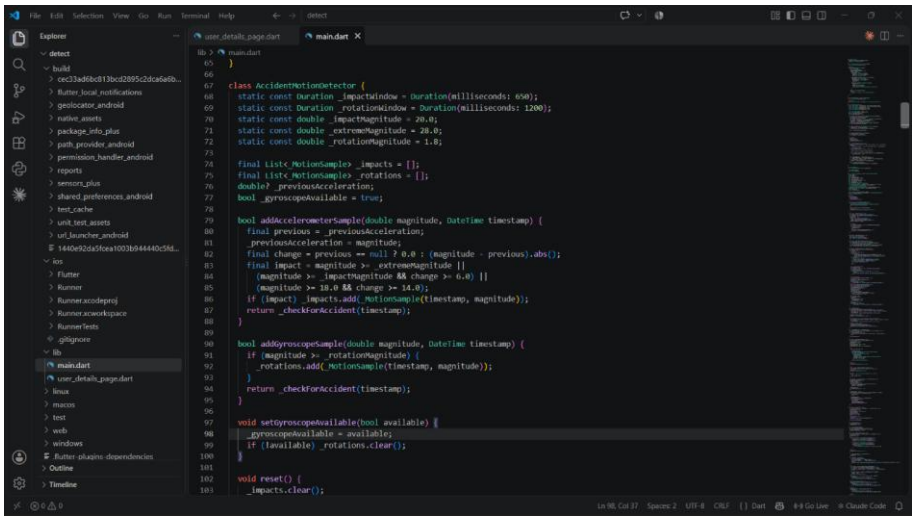
Figure 10.1: Source Code of the User Details Page (user_details_page.dart)

Figure 10.1 shows the beginning of the file `user_details_page.dart`, which implements the Emergency Details screen of Figure 5.1. It belongs to the User Details and Settings Module and is the code that runs when a user first enters the details or later chooses Edit Details. It is included so that the storage keys and the save behaviour described in Chapters 4 and 5 can be checked against the code.

The editor window shows the explorer panel on the left, with the project folder `detect`, its build-directory plug-in folders and the two files in `lib`, and the code on the right. The code imports the Flutter material library and the shared_preferences package, declares a stateful widget and creates a form key and three text-editing controllers for the name, blood group and number. The method `_load` reads the keys `user_name`, `blood_group` and `emergency_number` and uses empty text when they are missing. The method `_save` stops if form validation fails, writes the trimmed values with `setString`, checks that the widget is still mounted and closes the page.

When the page opens, `initState` calls `_load`, so previously saved details appear in the fields. When the user presses Save, the validation result decides whether anything is stored. The code therefore explains the persistence behaviour of Table 6.1 and the reason why the main screen can display the details without asking for them again.

10.1.2 Motion Detector Class



```
class AccidentMotionDetector {
  static const Duration _impactWindow = Duration(milliseconds: 650);
  static const Duration _rotationWindow = Duration(milliseconds: 1200);
  static const double _impactMagnitude = 20.0;
  static const double _extremeMagnitude = 28.0;
  static const double _rotationMagnitude = 1.8;

  final List<_MotionSample> _impacts = [];
  final List<_MotionSample> _rotations = [];
  double? _previousAcceleration;
  bool _gyroscopeAvailable = true;

  bool addAccelerometerSample(double magnitude, DateTime timestamp) {
    final previous = _previousAcceleration;
    _previousAcceleration = magnitude;
    final change = previous == null ? 0.0 : (magnitude - previous).abs();
    final impact = magnitude >= _extremeMagnitude ||
        (magnitude >= _impactMagnitude && change >= 6.0) ||
        (magnitude >= 18.0 && change >= 14.0);
    if (impact) _impacts.add(_MotionSample(timestamp, magnitude));
    return _checkForAccident(timestamp);
  }

  bool addGyroscopeSample(double magnitude, DateTime timestamp) {
    if (magnitude >= _rotationMagnitude) {
      _rotations.add(_MotionSample(timestamp, magnitude));
    }
    return _checkForAccident(timestamp);
  }

  void setGyroscopeAvailable(bool available) {
    _gyroscopeAvailable = available;
    if (!available) _rotations.clear();
  }

  void reset() {
    _impacts.clear();
```

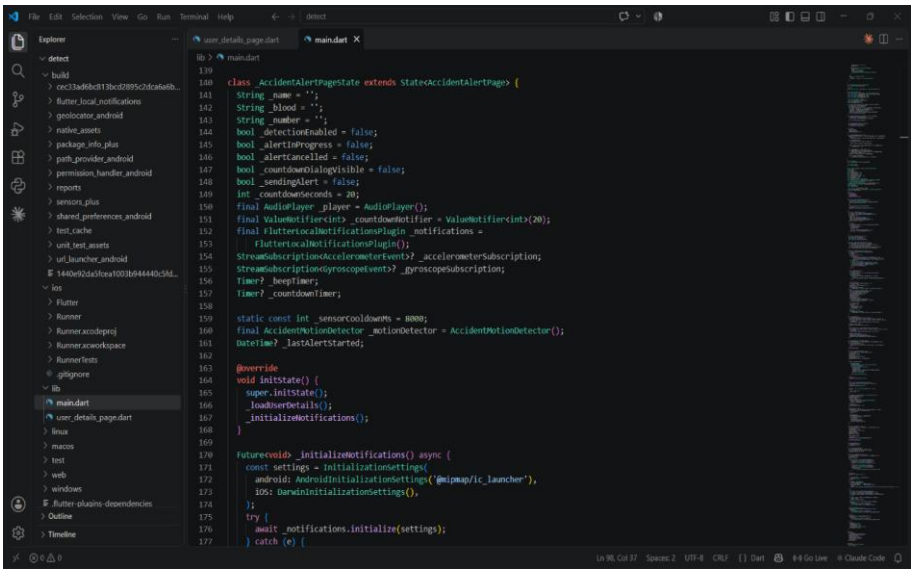
Figure 10.2: Source Code of the Motion Detector Class (main.dart, lines 65 – 103)

Figure 10.2 shows the class `AccidentMotionDetector` in `main.dart`. It is the part of the Sensor Monitoring Module that decides whether a sensor sample looks like an impact or a rotation, and it is called from the sensor listeners each time a reading arrives. It is the main source for the values of Table 5.3.

The top of the class declares the constants: an impact window of 650 milliseconds, a rotation window of 1200 milliseconds, an impact magnitude of 20.0, an extreme magnitude of 28.0 and a rotation magnitude of 1.8. Two lists, one for impacts and one for rotations, a field for the previous acceleration and a flag for gyroscope availability follow. The method `addAccelerometerSample` calculates the change from the previous magnitude, tests the three impact conditions, stores a qualifying sample and returns the result of `_checkForAccident`. The method `addGyroscopeSample` stores a rotation sample when the magnitude reaches the rotation limit. `setGyroscopeAvailable` clears the rotation list when the gyroscope is not available, and `reset` begins to clear the lists.

The detector returns a Boolean value to its caller, which treats it as a possible accident. The body of `_checkForAccident` lies outside the lines shown, so the exact rule that combines impacts and rotations is not described in this report. The figure shows how the prototype can run with the accelerometer alone, because the gyroscope list is cleared when that sensor is missing.

10.3.3 Alert Page State



```
class _AccidentAlertPageState extends State<AccidentAlertPage> {
  String _name = '';
  String _blood = '';
  String _number = '';
  bool _detectionEnabled = false;
  bool _alertInProgress = false;
  bool _alertCancelled = false;
  bool _countdownDialogVisible = false;
  bool _sendingAlert = false;
  int _countdownSeconds = 20;
  final AudioPlayer _player = AudioPlayer();
  final ValueNotifier<int> _countdownNotifier = ValueNotifier<int>(20);
  final FlutterLocalNotificationsPlugin _notifications =
      FlutterLocalNotificationsPlugin();
  StreamSubscription<AccelerometerEvent>? _accelerometerSubscription;
  StreamSubscription<GyroscopeEvent>? _gyroscopeSubscription;
  Timer? _beepTimer;
  Timer? _countdownTimer;

  static const int _sensorCooldownMs = 8000;
  final AccidentMotionDetector _motionDetector = AccidentMotionDetector();
  DateTime? _lastAlertStarted;

  @override
  void initState() {
    super.initState();
    _loadUserDetails();
    _initializeNotifications();
  }

  Future<void> _initializeNotifications() async {
    const settings = InitializationSettings(
      android: AndroidInitializationSettings('@mipmap/ic_launcher'),
      iOS: DarwinInitializationSettings(),
    );
    try {
      await _notifications.initialize(settings);
    } catch (e) {
```

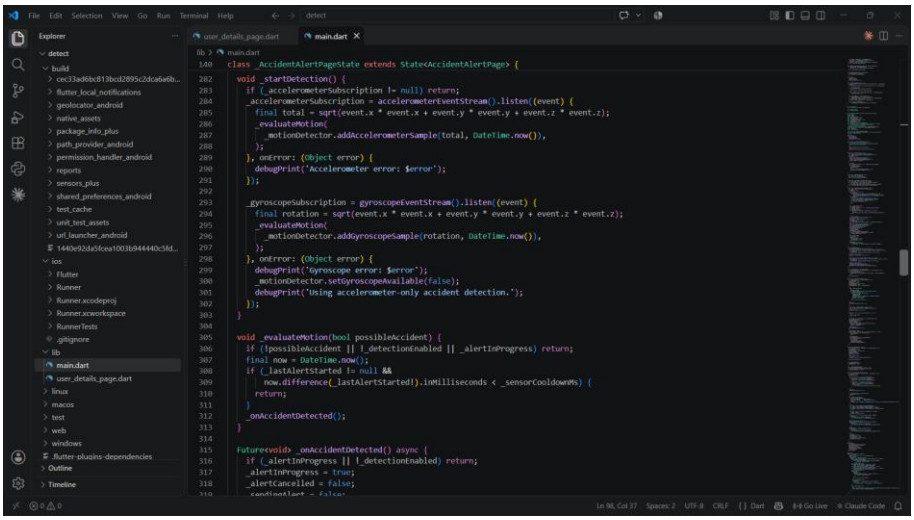
Figure 10.3: Source Code of the Alert Page State and Notification Set-up (main.dart, lines 139 – 177)

Figure 10.3 shows the start of the state class of the main Accident Alert screen of Figure 5.2. It belongs to the Detection Control and Alert Generation modules and defines the variables that hold the details and the state of the alert. It is included because it contains the 20-second countdown value and the 8-second cooldown constant that are referred to throughout the report.

The variables `_name`, `_blood` and `_number` hold the loaded details, and Boolean flags record whether detection is enabled, an alert is in progress, the alert has been cancelled, the countdown dialog is visible and a message is being sent. The countdown starts at 20, and a `ValueNotifier` with the value 20 lets the dialog update. An `AudioPlayer` and a local-notifications plug-in object are created, two stream subscriptions hold the accelerometer and gyroscope listeners, and two timers are declared for the beep and the countdown. The constant `_sensorCooldownMs` is 8000 and the motion detector object and the time of the last alert follow. The method `initState` loads the user details and initialises notifications using an Android launcher icon and default iOS settings inside a try block.

These variables show how the screen reacts to events: a change of a flag or of the notifier value causes the interface to be redrawn, and the flags are what prevent two alerts from running at once. The figure supports the statement that the implemented countdown is 20 seconds, and it is the evidence that local notifications are part of the application.

10.1.4 Sensor Subscriptions and Motion Evaluation



```
282  void _startDetection() {
283    if (_accelerometerSubscription != null) return;
284    _accelerometerSubscription = accelerometerEventStream().listen((event) {
285      final total = sqrt(event.x * event.x + event.y * event.y + event.z * event.z);
286      _evaluateMotion(
287        _motionDetector.addAccelerometerSample(total, DateTime.now()),
288      );
289    }, onError: (Object error) {
290      debugPrint('Accelerometer error: $error');
291    });
292
293    _gyroscopeSubscription = gyroscopeEventStream().listen((event) {
294      final rotation = sqrt(event.x * event.x + event.y * event.y + event.z * event.z);
295      _evaluateMotion(
296        _motionDetector.addGyroscopeSample(rotation, DateTime.now()),
297      );
298    }, onError: (Object error) {
299      debugPrint('Gyroscope error: $error');
300      _motionDetector.setGyroscopeAvailable(false);
301      debugPrint('Using accelerometer-only accident detection.');
302    });
303  }
304
305  void _evaluateMotion(bool possibleAccident) {
306    if (!possibleAccident || !_detectionEnabled || _alertInProgress) return;
307    final now = DateTime.now();
308    if (_lastAlertStarted != null &&
309        now.difference(_lastAlertStarted!).inMilliseconds < _sensorCooldownMs) {
310      return;
311    }
312    _onAccidentDetected();
313  }
314
315  Future<void> _onAccidentDetected() async {
316    if (_alertInProgress || !_detectionEnabled) return;
317    _alertInProgress = true;
318    _alertCancelled = false;
319    _sendingAlert = false;
```

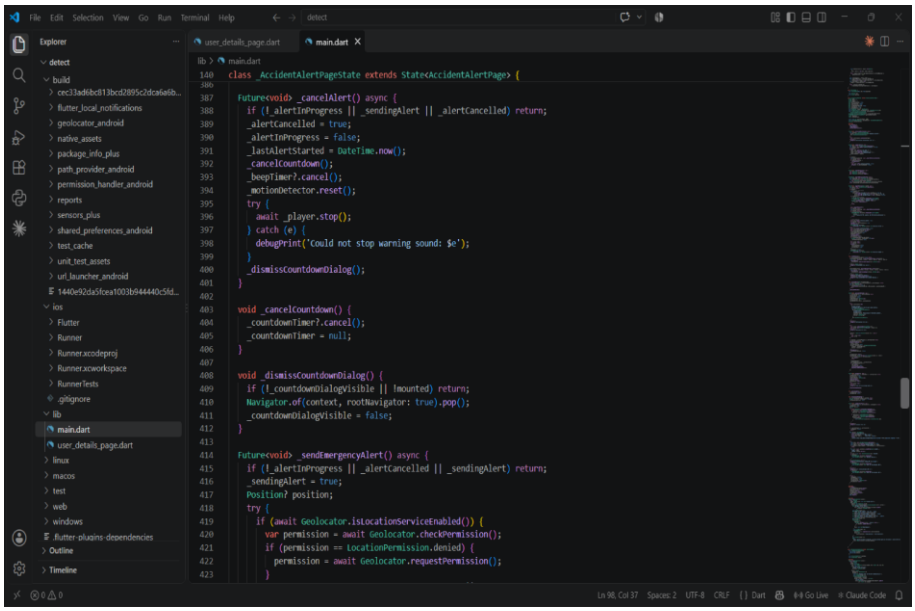
Figure 10.4: Source Code of Sensor Subscriptions and Motion Evaluation (main.dart, lines 282 – 319)

Figure 10.4 shows the method `_startDetection` and the method `_evaluateMotion`. This is the code that is executed when the user presses Start Detection in Figure 5.2, and it connects the sensor streams to the detector of Figure 10.2. It also contains the cooldown check, which makes it the key reference for the Sensor Monitoring Module.

In `_startDetection` the code returns if an accelerometer subscription already exists, so detection cannot be started twice. It then listens to the accelerometer event stream, calculates `total` as the square root of the sum of the squares of the three axis values, and passes it with the current time to `addAccelerometerSample` inside a call to `_evaluateMotion`. An error handler prints a message. The gyroscope stream is handled in the same way with a variable named `rotation`; its error handler marks the gyroscope as unavailable and prints "Using accelerometer-only accident detection." The method `_evaluateMotion` returns if there is no possible accident, if detection is disabled or if an alert is in progress, then returns if less than the cooldown has passed since the last alert, and otherwise calls `_onAccidentDetected`, whose first lines are visible at the bottom.

Together these lines show the order in which the conditions are applied: sensor sample, detector result, enabled and idle state, cooldown and then the alert. They explain why repeated shakes produce only one alert (Table 6.2, case 8) and why disabling detection prevents new alerts (Table 6.2, case 2). The first lines of `_onAccidentDetected` set the alert flags before the alarm and countdown begin.

10.1.5 Cancellation and Emergency Escalation



```
class _AccidentAlertPageState extends State<AccidentAlertPage> {

  Future<void> _cancelAlert() async {
    if (!_alertInProgress || _sendingAlert || _alertCancelled) return;
    _alertCancelled = true;
    _alertInProgress = false;
    _lastAlertStarted = DateTime.now();
    _cancelCountdown();
    _beepTimer?.cancel();
    _motionDetector.reset();
    try {
      await _player.stop();
    } catch (e) {
      debugPrint('Could not stop warning sound: $e');
    }
    _dismissCountdownDialog();
  }

  void _cancelCountdown() {
    _countdownTimer?.cancel();
    _countdownTimer = null;
  }

  void _dismissCountdownDialog() {
    if (!_countdownDialogVisible || !mounted) return;
    Navigator.of(context, rootNavigator: true).pop();
    _countdownDialogVisible = false;
  }

  Future<void> _sendEmergencyAlert() async {
    if (!_alertInProgress || _alertCancelled || _sendingAlert) return;
    _sendingAlert = true;
    Position? position;
    try {
      if (await Geolocator.isLocationServiceEnabled()) {
        var permission = await Geolocator.checkPermission();
        if (permission == LocationPermission.denied) {
          permission = await Geolocator.requestPermission();
        }
```

Figure 10.5: Source Code of Alert Cancellation and the Start of Escalation (main.dart, lines 387 – 423)

Figure 10.5 shows the methods `_cancelAlert`, `_cancelCountdown`, `_dismissCountdownDialog` and the beginning of `_sendEmergencyAlert`. They implement the false-alert cancellation and the first steps of escalation described in Section 5.2.4 and Section 5.2.5, and they are included to show how the application stops an alert and how it checks location access.

The method `_cancelAlert` returns unless an alert is in progress and has not already been cancelled or sent. It sets the cancelled flag, clears the in-progress flag, records the current time as the last alert, cancels the countdown and beep timers, resets the motion detector, stops the audio player inside a try block that logs a failure, and closes the countdown dialog. `_dismissCountdownDialog` pops the dialog only if it is visible and the widget is still mounted. `_sendEmergencyAlert` returns unless an alert is in progress and not cancelled, sets the sending flag, and begins a try block that checks whether the location service is enabled, checks the permission and requests it when it has been denied.

When the user cancels, no further step of `_sendEmergencyAlert` can run, because its first guard checks the cancelled flag; this is the code behind the test case in which cancellation stops the alarm and no message is generated (Table 6.2, case 5 and Table 6.3, case 3). The permission check explains the permission-denied test case (Table 6.3, case 5). The remainder of the method, in which the position and the message are produced, is not visible in the supplied screenshot.

CHAPTER 11

REFERENCES

[1] Aloul, F. A., Zualkernan, I. A., Abu-Salma, R., Al-Ali, H., & Al-Merri, M. (2015). iBump: Smartphone application to detect car accidents. Computers & Electrical Engineering, 43, 66-75. https://doi.org/10.1016/j.compeleceng.2015.03.003

[2] Android Developers. (2026). Sensors overview and SensorManager API documentation. Android Developer Documentation.

[3] Bhatti, F., Shah, M. A., Maple, C., & Islam, S. U. (2019). A novel Internet of Things-enabled accident detection and reporting system for smart city environments. Sensors, 19(9), 2071. https://doi.org/10.3390/s19092071

[4] Dart. (2026). Dart language documentation. https://dart.dev/

[5] Fanca, A., Puscasiu, A., & Valean, H. (2016). Accident reporting and guidance system with automatic detection of the accident. 20th International Conference on System Theory, Control and Computing. https://doi.org/10.1109/ICSTCC.2016.7790657

[6] Fernandes, B., Alam, M., Gomes, V., Ferreira, J., & Oliveira, A. (2016). Automatic accident detection with multi-modal alert system implementation for ITS. Vehicular Communications, 3, 1-11. https://doi.org/10.1016/j.vehcom.2015.11.001

[7] Fernandes, B., Gomes, V., Ferreira, J., & Oliveira, A. (2015). Mobile application for automatic accident detection and multimodal alert. IEEE Vehicular Technology Conference (VTC Spring). https://doi.org/10.1109/VTCSpring.2015.7145935

[8] Flutter. (2026). Flutter documentation. https://docs.flutter.dev/

[9] Geolocator package. (2026). Flutter geolocation plugin documentation. https://pub.dev/packages/geolocator

[10] Google Maps Platform. (2026). Maps URLs documentation. Google Developers.

[11] International Telecommunication Union. (2024). Intelligent transport systems and emergency communication resources. ITU.

[12] Lahn, J., Peter, H., & Braun, P. (2015). Car crash detection on smartphones. Proceedings of the 2nd International Workshop on Sensor-based Activity Recognition and Interaction. https://doi.org/10.1145/2790044.2790049

[13] Lane, N. D., Miluzzo, E., Lu, H., Peebles, D., Choudhury, T., & Campbell, A. T. (2010). A survey of mobile phone sensing. IEEE Communications Magazine, 48(9), 140-150.

[14] Permission Handler package. (2026). Flutter permission_handler plugin documentation. https://pub.dev/packages/permission_handler

[15] Quick Accident Detection and Response System. (2020). Prototype proposal for quick accident detection and response system. 5th International Conference on

Research in Computational Intelligence and Communication Networks. https://doi.org/10.1109/ICRCICN50933.2020.9296153

[16] ResQ - Accident Detection and Alerts App. (2025). Utilitas Mathematica.

[17] Roy, R. (2021). Flutter Mobile Development for Beginners. Packt Publishing.

[18] Sensors Plus package. (2026). Flutter sensors_plus plugin documentation. https://pub.dev/packages/sensors_plus

[19] Shared Preferences package. (2026). Flutter shared_preferences plugin documentation. https://pub.dev/packages/shared_preferences

[20] URL Launcher package. (2026). Flutter url_launcher plugin documentation. https://pub.dev/packages/url_launcher

[21] Vehicle Accident Detection and Locating Using GSM and GPS. (2024). International Conference on Communication, Computing and Internet of Things (IC3IoT). https://doi.org/10.1109/IC3IoT60841.2024.10550407

[22] White, J., Thompson, C., Turner, H. A., Dougherty, B., & Schmidt, D. C. (2011). WreckWatch: Automatic traffic accident detection and notification with smartphones. Mobile Networks and Applications, 16, 285-303. https://doi.org/10.1007/s11036-011-0304-8

[23] World Health Organization. (2023). Global status report on road safety 2023. WHO.

[24] World Health Organization. (2023). Road traffic injuries: Key facts. WHO.

[25] Yadav, S. et al. (2021). A survey on IoT based automatic road accident detection. 5th International Conference on Intelligent Computing and Control Systems (ICICCS). https://doi.org/10.1109/ICICCS51141.2021.9432159

Abhishikth S Siby

Accident_DetectionReport_Final22

mini project report

Document Details

Submission ID
trn:oid:::10159:154239045

Submission Date
Oct 10, 2026, 11:12 PM GMT+5:30

Download Date
Oct 12, 2026, 8:10 PM GMT+5:30

File Name
Accident_DetectionReport_Final22.pdf

File Size
1.9 MB

76 Pages

21,677 Words

122,963 Characters

13% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Match Groups

157 Not Cited or Quoted 12%
Matches with neither in-text citation nor quotation marks

5 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

9% Internet sources
7% Publications
10% Submitted works (Student Papers)